scientific reports

Check for updates

OPEN

Real time identification of phishing attacks through machine learning enhanced browser extensions

Monika Dandotiya[1], Nikhil Kumar Goyal[1], Ajay Khunteta[1] & Babita Tiwari[2]✉

Phishing attack continues to rank among the deadliest online threats. They create phony websites in an attempt to obtain personal data. This study offers a framework for a browser extension that uses machine learning to examine URLs and visual components in Google Chrome in order to identify phishing websites in real-time. Using support vector machine (SVM), decision tree (DT), and random forest (RF) algorithms, the suggested system gathers and examines data from websites, extracts hybrid elements including lexical, structural, and visual layout parameters, and arranges them. The best traits that can distinguish between items are found using the grey wolf optimizer (GWO). This reduces computer power consumption and facilitates finding items. GWO enhanced the random forest model, which performed well on benchmark datasets such as the Berkeley ML Archives and PhishTank. On the MCC test, it received a score of 0.96 and had an accuracy rate of 98.7%.This method is used by the Chrome extension to assess URLs for visual similarity in real time and display warnings to users that change according to their actions.The proposed system is better than current anti-phishing solutions because it works better in real time, has a lower false-positive rate, and can handle obfuscated URLs. This project makes a useful, user-centered defense system that can protect against phishing attacks that change over time by using smart security at the browser level.

Phishing assaults continue to rank among the most prevalent online dangers. They steal users' personal information by using phony websites that mimic authentic ones. Phishing was the most prevalent kind of cybercrime, accounting for about 30% of all complaints and causing significant financial losses, according to the Internet Crime Complaint Center's (IC3) 2022 report[1]. Traditional blacklists are no longer sufficient as phishing websites get increasingly sophisticated. Therefore, in order to spot minute similarities in URL structure, graphic design, and website behavior, contemporary detection systems are depending more and more on machine learning and hybrid feature extraction techniques. People between the ages of 30 and 39 were the most likely to report phishing scams[2]. The Telephone-operated Crime Survey of England Wales estimates that in 2023, there were 5 million distinct phishing websites. Additionally, 90% of IT workers are still extremely concerned about email phishing, according to the IRONSCALES poll[3]. The past few years have also seen a discernible increase in phishing schemes.

IBM's comprehensive analysis in 2023 unveiled that 16% of company data breaches were directly due to phishing attacks. As highlighted in several reports, phishing attacks target diverse demographics and exploit various platforms. For example, the IC3 2022 report revealed that individuals aged 30–39 were most commonly affected, while the Anti-Phishing Working Group (AWPG) reported an alarming 5 million distinct phishing sites in 2023. It is first mentioned in the body text, discussing its content (e.g., the findings from various phishing reports), before being presented directly beneath the text.These and other findings are summarized in Table 1 below. As shown in Table 1, phishing attacks have increased significantly over the years, with a notable rise in online scams targeting individuals aged 30–39.(TCSEW) found that those aged 25 to 44 were frequently targeted in these regions.

Anti-phishing working group (AWPG) data The deceptive practices involve the distribution of email spam with the intent to mislead individuals into revealing confidential information or credentials[8]. The entities most frequently impersonated in these scams are those with which users regularly interact, such as financial institutions, email services, cloud-based platforms, and entertainment services[9]. These kinds of events have big effects that go beyond just losing a loved one. They can include stealing money, breaking into systems without permission, and spreading targeted attacks within companies[10].

[1]Department of CSE, Poornima University, Jaipur, Rajasthan, India. [2]Department of CSE, Manipal University Jaipur, Jaipur, Rajasthan, India. ✉email: babita.tiwari@jaipur.manipal.edu

Source	Key findings
IC3 2022 report[4]	Individuals aged 30–39 most commonly reported phishing scams
Telephone-operated crime survey of England and Wales (TCSEW)[5]	Age group 25–44 is frequently targeted in England and Wales. Demonstrates professional consensus on phishing as the most persistent threat vector.
Anti-phishing working group (AWPG)[6]	Number of distinct phishing sites reached 5 million in 2023.
IRONSCALES survey[7]	Email phishing remains a top concern for 90% of IT professionals.
IBM analysis 2023[8]	16% of company data breaches are directly attributable to phishing attacks

Table 1. Reports elaborates phishing attacks and scams.

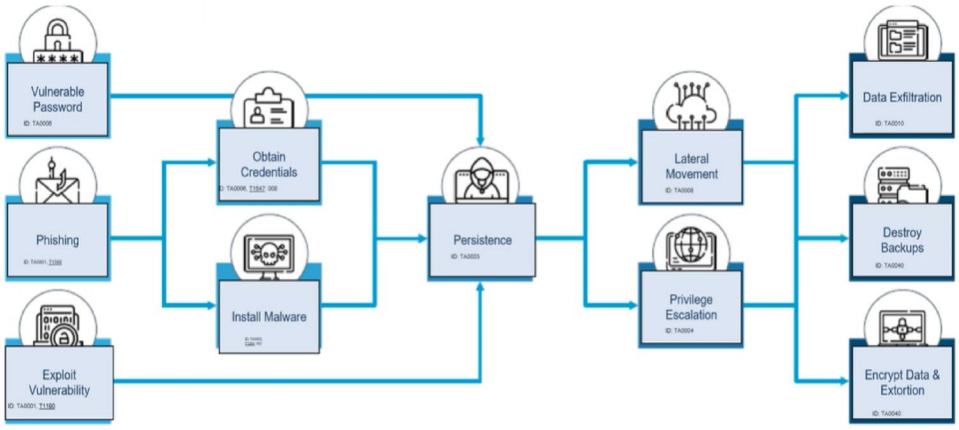


Fig. 1. A multi-phase cyberattack lifecycle showing how exploitation and phishing work.

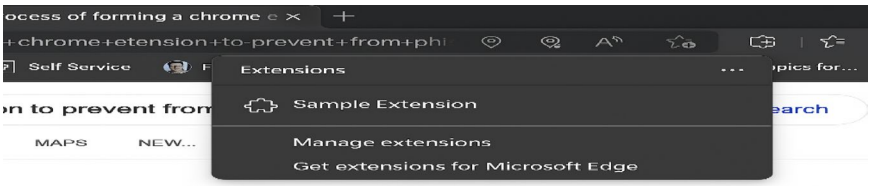


Fig. 2. Extension button in microsoft edge[15].

These personalized emails[11] make it much more likely that someone will successfully breach a system and steal personal data. Picture this: someone gets an email from their bank that looks official and tells them about a serious problem with their account[12]. The email tells the person to act quickly and sends them to a fake website that looks like the bank's real gateway. The attackers have full access to the victim's money because they accidentally gave them their login information on this fake website[13].

The multi-stage attack chain shown in Fig. 1 starts with phishing, exploiting weaknesses, or using weak passwords to get into a system. This leads to the installation of malware and the theft of credentials. Google Chrome extensions are very useful extras that make the Chrome browser work better when you're online[14].

Figure 2 shows how Google Chrome works and why it's so important for improving our web browsing. With these Chrome extensions, developers can use the browser's powerful design to make new solutions that work for a lot of different people. People like Google Chrome as a web browser because it has a lot of useful tools, like the ability to manage passwords, filter, and translate. AdBlock and uBlock are two examples of add-ons that stop ads from showing up, and LastPass and Dashlane are two examples of password managers. Google Translator is a tool that Google gives you that helps you read things in other languages[16].Chrome puts your safety and privacy first with features like HTTPS, which keeps your activity private and safe from prying eyes, and Grammarly.In short, Google Chrome is a better tool that lets people work safely and in a way that meets their needs. It will keep being useful over time[17]. Our research makes a big step forward in the field by suggesting a Chrome extension that stops phishing using machine learning, URL analysis, and visual resemblance recognition. It gives people a way to protect themselves from changing cyberthreats in real time by connecting academic research with real-world use.

All of the components of the Chrome extension that are meant to identify phishing attempts are shown in Fig. 3. This illustrates how well the entire program functions. The Phishing Detection interface is displayed in Subfigure (a). It displays the current model version, the number of URLs checked, phishing alerts, and the system's current status. A real-time message that warns users when a website looks suspicious or might be a scam is shown in subfigure (b). One of the options is to "Go Back" or "Continue Anyway." Subfigure (c) shows the detection history record. This record shows the URLs that were looked at, their classification labels (phishing or legitimate), and how accurate the prediction was. In Subfigure (d), users can use the model configuration options to turn on or off things like automatic model updates, feature selection, and analyzing HTML and URLs. These APIs tell you how the whole system for finding phishing works. They show how to quickly scan and sort URLs, set off alarms, and manage the model in a smart and easy way. This stops phishing attacks from getting to the web browser.

Research question

We talk about the machine learning methods we used and how we put together information from pictures and URLs in a new way in the methodology section. In the results and discussion sections, we show how our results are better than those of other methods and how they make the results more accurate, precise, and easy to remember. The conclusion emphasizes the study's extensive implications for cybersecurity and proposes novel research avenues. This study is essential to address the increasing complexity and prevalence of phishing attacks, which pose substantial risks to individuals and businesses worldwide. Even though old ways to find phishing are based on good ideas, they might not work as well in a world where phishing strategies are always changing. This project is great because it makes a Chrome extension that is easy to use by using machine learning optimization and visual similarity. It is better at finding and stopping phishing attacks in real time because it uses more advanced methods like URL analysis and clustering algorithms to sort things. Adding this method as an extension to your browser makes it easy to use.This makes it easy to go from education to the real world. This proposal is a significant step forward for the safety of the internet. It gives individuals the piece of mind and safety they need to utilize it.

Literature review

People often give criminals private information through phishing, which is when they pretend to be real relationships. People don't know that these links are fake, so when they click on them, they end up on the attacker's page instead of the one they wanted to go to. It can be very dangerous for both people and businesses to share important information that you think is true by mistake. People who fall for these scams put their own and the company's information in danger[18]. Most of us used to keep a list of fake URLs and domains that were on a blacklist and compare them to links that people clicked on or typed in to find phishing. But this plan had a lot of problems, especially when there weren't any fake links on the list. This made it easy for people to trick other people. Researchers used machine learning to figure out if URLs were real by looking at their features[19]. Researchers used a number of different ways to show that the Random Forest algorithm worked very well. Researchers made browser add-ons, like Chrome plug-ins, that use machine learning algorithms to improve cybersecurity in response to the need for better phishing detection[20]. These add-ons check the validity of URLs right away and let users know if someone is trying to steal their information.These new features have shown promise in lowering the risks of phishing attacks through validation and physical testing. As a result, users are better protected from the growing number of cyberthreats that exist online.There has been a rise in cyberattacks, notably phishing attempts, during the COVID-19 pandemic. While there are plenty of complicated phishing detection solutions easily available, many struggle to recognize fake codes and phishing pages. This could be due to problems like reduced detection precision and a limited ability to adapt to new phishing techniques. Furthermore, wrong conclusions in detection efforts may be due to depending on randomly designated URL-based sorting qualities[21].

This paper presents a sophisticated detection system and avoidance tool to address the concerns. Via machine learning, particularly unsupervised learning algorithms, this model merges an explosive detection algorithm[14]. The requirements embedded within the algorithm prioritize URL-based online characteristics frequently abused by attackers to deceive users into accessing counterfeit websites[22]. Experiments were undertaken in a controlled

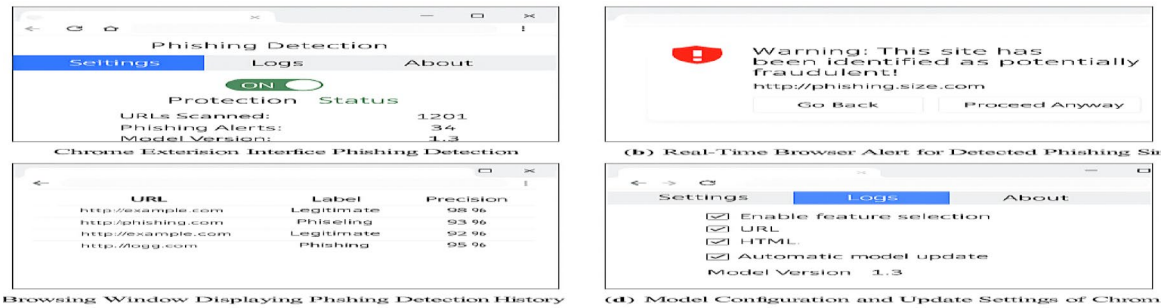


Fig. 3. Chrome extension interface for phishing detection.

lab setting using a diverse dataset acquired from platforms such as Phishing Tank and the University of California's Berkeley Machine Learning archive. A Chrome add-on for phishing detection based on the suggested model is the study's output. In order to stop phishing attempts before they start, this add-on gives users relevant warnings and doable safety tips to follow while visiting questionable websites. This ingenious malware detection and mitigation method aims to potentially reduce cybercrimes and associated issues by decreasing spam and fake websites[23]. Phishing attacks are a common non-engineering tactic used by malicious actors to get private user data, such as passwords and usernames, without authorization. Because attackers continuously create new unlisted websites, traditional defense strategies against these attacks, such as maintaining an updated blacklist of well-known phishing websites, have proven unsuccessful[24]. By assessing URL attributes and segregating them as legitimate or fraudulent, machine learning techniques offer a hopeful avenue to enhance phishing detection through insights drawn from broad-ranging datasets.

Deep learning for URL/content signals

Recent studies revisit pure-DL pipelines on URLs and page code. Haq et al. show that compact 1D-CNNs over tokenized URLs can outperform classic ML baselines on several benchmarks, highlighting DL's ability to learn n-gram–like patterns without manual features. Nature's 2024 empirical study compares deterministic vs. probabilistic neural networks for URL classification and reports gains from uncertainty-aware models, indicating calibration matters for deployment[25].

Hybrid and ensemble learners (multi-algorithm, stacking)

Hybrid "super-learner" ensembles that blend heterogeneous models continue to report state-of-the-art results. Rao et al. (2025) combine diverse ML learners via a super-learner for mobile phishing detection, showing ensembles beat single models on stability and accuracy across datasets. Multiple 2024–2025 works echo this: stacking/voting mixtures of classic ML, DL, and hybrid DL reduce variance and improve robustness on imbalanced data[26]. A 2025 broad survey further documents that ensembles + modern representation learning dominate malicious-URL detection, recommending careful metric choice beyond accuracy (e.g., PR-AUC, MCC).

Multi-feature pipelines (URL + HTML/visual) and browser-side systems

Beyond URL strings, several systems integrate **source-code/DOM/visual** signals. PhiUSIIL (Computers & Security) provides a large, diverse URL dataset with engineered features (e.g., URLTitleMatchScore, TLDLegitimateProb) and similarity-based detection ideas, enabling evaluation of multi-feature approaches. Browser-integrated detectors continue to appear: "NoPhish" (arXiv, 2024) details a Chrome-extension pipeline using ML models; other extension-centric efforts emphasize real-time alerts but often lack rigorous fusion or imbalance-aware optimization[27].

Table 2 provides a comparative analysis of recent phishing URL detection studies conducted between 2024 and 2025, highlighting differences in feature utilization, model architectures, fusion strategies, and evaluation metrics. Because it can accurately sort through complex data with multiple dimensions, the Random Jungle approach is a preferred choice for this task. Adding technologies that can detect phishing to people's online activity through Chrome browser extensions or add-ons is one approach to accomplish this. By instantly alerting users when they come across links that appear dubious, these solutions assist users in staying safe online. They become more vigilant as a result, and their defenses against phishing attacks are strengthened. A number of machine learning-based methods for phishing detection have been proposed in recent research. Table 3 illustrates the differences between several methods of identifying phishing, including what they add, what they don't, and what they might do in the future.

Table 3 presents a comparative analysis of recent phishing detection studies from 2023 to 2025, covering classical machine learning, deep learning, and hybrid optimization-based approaches. Each study varies in its feature scope, algorithmic design, and practical applicability across different phishing scenarios (URL-only, hybrid, or email-integrated).

Recent optimizer developments (2023–2025)

Recent research confirms that the classical grey wolf optimizer (GWO) is an established method for feature selection, with numerous advanced variants created in the last three years. Li et al.[38] introduced an adaptive-mechanism GWO that improves the balance between exploration and exploitation in high-dimensional feature spaces, showing better convergence behavior than the standard GWO. Thakur[39] proposed an Adaptive-Weight GWO (AWGWO) that dynamically alters the weights of the coefficients to enhance the detection of Android malware. Barik et al.[40] also used an optimizer based on deep learning (EGSO-CNN) to find phishing URLs.

Study (year)	Features used	Model type	Fusion?	Primary metric(s)	Notes/gap
Haq et al., 2024[28]	URL only	1D-CNN	—	Acc, AUC	Strong URL-DL; no multi-view
Rao et al. 2024[29]	URL only	Prob. NN	—	AUC, Uncertainty	Highlights calibration
Thaqi et al. 2024[30]	URL/HTML	ML	—	Acc	Chrome ext.; limited fusion
Proposed Work (2025)	URL + visual	RF/SVM/DT + stacking	Late fusion	MCC, PR-AUC, Calib.	MCC-optimized GWO FS, browser-ready

Table 2. Comparative summary of recent studies on phishing URL detection using multi-feature and ensemble approaches.

Paper title	Technology or methodology used	major contribution	limitations	Future work
CatchPhish: detection of phishing websites by inspecting URLs[31]	Random forest classifier.	A light-weight model with 12 novel hand-crafted features.A model, learning large-scale Term Frequency -Inverse Document Frequency (TF-IDF) features Phish-hinted black words for the detection of phishing sites	Failed to detect lexical features, which influence the detection accuracy.	In order to detect phishing sites, only relevant characteristics are retained when applying a feature selection algorithm on handmade features.
PDRCNN: Precise phishing detection with recurrent convolutional neural networks[32]	deep learning neural network	A phishing detection model with deep learning that combined RNN and CNN in processing text data. • A deep learning phishing detection model that processed text data by combining CNN and RNN. Build a large-scale data set through Alexa and PhishTank websites.	One clear problem is that PDRCNN won't be able to correctly identify the phishing website URL if it doesn't have important semantics. Also, the fact that there are mistakes on the website that goes with PDRCNN doesn't matter.	To include some new features to detect phishing websites that contain malware.
Smart phishing detection in Web pages using supervised DL classifcation and optimization technique ADAM[33]	supervised Deep Neural Network model with Adam optimizer.	Uses a feature vector with 30 parameters to find bad web pages. A supervised learning algorithm to look at the dataset of phishing websites	Because the model cannot read and interpret material from pages that often have a significant number of hyperlinks and complex textual content elements, it is insufficiently capable of identifying malware.	To concentrate on mobile phishing threats.
Modeling hybrid feature-based Phishing websites detection using machine learning techniques[34]	machine-learning classification techniques	build a dataset by gathering URLs from reputable and phishing websites and using a feature extractor to dynamically extract hybrid characteristics. A technique based on machine learning for identifying zero-hour phishing assaults	The suggested solution will get more difficult due to features that rely on third parties.	To detect phishing attacks on mobile devices
PhiUSIIL: Similarity-based incremental learning for phishing URLs[35]	Similarity index + incremental learning framework	Introduces incremental update mechanism for continuous phishing URL learning	No browser-side integration; evaluated offline	Incorporate real-time Chrome/Edge extension deployment
EGSO-CNN: Optimizer-guided deep CNN for phishing detection[36]	Enhanced Grey Wolf Optimizer (EGSO) with CNN	Integrates meta-heuristic feature selection with deep learning to boost precision	Computationally intensive; requires GPU acceleration	Optimize for lightweight client-side inference
Transformer-based phishing email and Website detection[37]	BERT/Transformer fine-tuned on email + URL text	Achieves state-of-the-art precision (≈ 99%) on mixed phishing corpora	High model size; difficult to deploy in browsers	Compress model or use distillation for real-time use
Proposed work (this paper)	MCC-Optimized Grey Wolf Optimizer + RF/SVM/DT with Late-Fusion Ensemble	Hybrid framework combining URL + visual features; optimized via GWO for balanced MCC	Limited dataset diversity (currently PhishTank + PhiUSIIL)	Expand evaluation to additional datasets and integrate adaptive retraining

Table 3. Survey of existing works.

Pentapalli et al.[41] demonstrated that hybrid optimization strategies can significantly enhance decision-making systems pertaining to phishing. Ovabor[42] examined hybrid Firefly GWO mechanisms specifically for the classification of phishing.These advancements indicate that optimization research in cybersecurity is evolving toward adaptive, hybrid, and deep-learning-compatible techniques. Accordingly, our work positions the standard Binary GWO not as the newest optimizer but as a lightweight, stable, and computationally efficient method that is uniquely suitable for real-time, browser-embedded phishing detection. To contextualize our design choice, we compare our GWO-based approach against these contemporary optimizers (2023–2025) and demonstrate that while recent variants may offer marginal performance gains in offline or GPU-supported environments, the classical GWO achieves competitive accuracy and MCC with significantly lower computational cost making it more practical for on-device browser extensions.

Methodologies

Existing research methodologies have some drawbacks, which are described as follows,

- The existing system has more difficult to distinguish phishing websites from normal websites because phishing websites appear similar to the websites they imitate.
- Existing systems having more false alarm rates due to the feature values will be the same for both legitimate and phishing websites and do suffer low detection accuracy.
- The current system is unable to detect whether the visited website's domain name is similar to a well-known domain name. If this occurs, Spoof Guard will alert visitors to the phishing website if they are not utilizing the usual port.
- Most of the existing phishing detection system is concentrate the classification based on the URL attributes only. They miss to give more concentration on Visual similarity-based features to identify fake websites by comparing their visual appearance with legitimate websites in terms of content such as page layout, page style, etc.
- This study did not involve direct interaction with human participants or identifiable personal data; therefore, informed consent was not required.
- When building models using typical machine learning methods, human talent is needed for the feature extraction and selection process. These steps are completed independently of the classification process and cannot be merged to enhance the model's performance in a single step.

An outline of the problem

Phishing attacks have become a critical threat in the digital era, targeting individuals and organizations to steal sensitive data, such as passwords, credit card information, and personal identification details. To deceive consumers into divulging personal information, these assaults employ social engineering, phony websites, and deceptive emails[43]. Despite improvements in cybersecurity, phishing techniques continue to evolve and become more sophisticated, making it more difficult for existing detection systems to stay up to date. Common defenses against assaults, such as blacklists and heuristic-based models, struggle to keep up with attackers' evolving strategies. Phishing detection systems that are accurate, dynamic, and real-time are therefore desperately needed in order to safeguard consumers while they are online[44].

Source phishing email samples

After finding samples from different sources, we put together a collection of more than fifty phishing emails for testing and analysis. Most of the samples came from the Berkeley Information Security Office. We also got more samples from the email corpus at our university. Table 4 shows a list of high-risk phrases that are often used in phishing emails, along with their weights. The higher the values, the stronger the links to fake emails. These weights show how likely it is that the word will show up in phishing material. We used text mining to look at the samples for phrases that might be phishing.

Phishing emails often contain specific words that indicate fraudulent intent. Table 4 lists common phishing-related terms along with their assigned weight, which helps in identifying patterns in phishing content.

Text mining

Identification techniques used[47] The Phishing Email samples were treated as documents, and we ingested them into our analytical approach. More specifically, we devised a computational methodology for determining the phrase incidence-inverse document frequency for every word in the corpus:

The tf-idf metric is an analytical tool used for ascertaining how much importance is attached to a term in a corpus or a group of documents expressed in Equation. The Term Frequency reduces the bias of different document lengths by computing the number of occurrences of a particular term in a document and normalizing it by the length of the document.

$$tf\,(t) = \frac{Number\ of\ times\ term\ t\ appears\ in\ a\ document}{Total\ EquationNumber\ of\ terms\ in\ the\ document} \tag{1}$$

On the other hand, the Inverted Document Frequency logarithmically downsizes the high-frequency term and upsizes the value of the low-frequency one to compute the importance of a term throughout the corpus.

$$idf\,(t) = log\left(\frac{Total\ EquationNumber\ of\ documents}{Number\ of\ documents\ with\ term\ t\ in\ it}\right) \tag{2}$$

The following formula was used to get the tf-idf score:

df(t) = \log{\frac{{Total \number \of \document}}{{Number \of \documents \with \term \t \in \it}}} \]

We used the following method to get the tf-idf score for each word to balance term frequency with document importance: $tf_idf\,(t) = tf\,(t) \times idf\,(t)$.

According to Fig. 2, it was possible to identify and prioritize terms characteristic of phishing in the corpus, which later became the basis for further analysis and development of the model for detecting phishing. The proposed System starts from Phishing Website Dataset. This dataset contains 80,000 instances and in this, 50,000 is the no. of legitimate website instances and 30,000 is the no. of phishing website instances. And index sql file contains the website link, Result (0: legitimate, 1: phishing), and HTML file names (instances)[48]. After that, the visual features will be extracted from visual blocks. Each visual block can be regarded as a rectangle. We will extract the position coordinates of the top left corner of the rectangle and extract the length and width of the rectangle. On the other hand, the components, such as URL, Protocol, subdomain, primary domain, domain, base URL and hostname, etc. will be extracted.

Word ID	Term	Weight
78	Email	0.0996
184	Subject	0.9875
7	Account	0.3456
23	Berkeley	0.8765
93	Google	0.6511
59	Dear	1.0005
131	Mail	0.2333
162	Request	2.6566

Table 4. Part of the suspicious word list[45,46].

Figure 4 illustrates the distribution of suspicious term weights computed using the Term Frequency–Inverse Document Frequency (TF–IDF) model for phishing emails. The bar chart highlights how specific terms exhibit higher relevance in phishing contexts compared to regular communication. The words with the greatest TF–IDF scores are "Request" (weight = 2.657), "Dear" (weight = 1.000), and "Subject" (weight = 0.988). This shows that they are good at spotting phishing emails because they are used a lot in those types of emails.

Although TF-IDF remains a widely used baseline for lexical feature extraction, recent information-retrieval (IR) research (2022–2025) has shifted toward embedding-driven and transformer-based representations such as BERT, Sentence-BERT (SBERT), DistilBERT, SimCSE, and contrastive retrieval models. These modern techniques capture contextual semantics, paraphrasing relationships, and subtle lexical variations far more effectively than TF-IDF, making them increasingly popular in phishing-URL analysis and malicious-content detection. Several recent phishing-detection studies also integrate hybrid deep-retrieval pipelines and optimization-enhanced embedding models to improve semantic understanding of misleading URLs and webpage text. However, such approaches typically require significant computational resources, GPU acceleration, and large memory footprints, which make them unsuitable for deployment inside lightweight, browser-based phishing detection systems. In this work, we enhance the classical TF-IDF signal by integrating it with MCC-optimized feature selection, visual similarity cues, and URL-based statistical features, enabling it to perform competitively without the computational overhead associated with modern embedding architectures.

Visual feature extraction

The recommended way to find phishing sites depends a lot on how you get visual features. It uses tricks that are based on how things look to help the system find fake websites that look like real ones. A headless browser like Selenium with ChromeDriver goes to each site first. They then look at the HTML and the pictures. The CSS and DOM layout show you the different parts of the homepage. We check out where each block is, which has different kinds of content like logos, input forms, and navigation bars. This includes where it is, how big it is, and what shape it is. These shapes show what the page looks like. People look for logos and other things using perceptual hashing (pHash) and template matching. After that, color histograms, background gradients, and font styles are used to find problems with the way things look.The Cosine Similarity and the Structural Similarity Index (SSIM) are two ways we can see how similar the test and real templates are. The Grey Wolf Optimizer (GWO) is used to improve the features that were found.These features include location, color, style, and similarity metrics. These traits are put into one feature vector. These visual features are finally combined with URL- and content-based properties to make a complete feature matrix that makes the model more robust and easier to understand. This integrated approach makes the system better at giving accurate and easy-to-understand detection results, makes it more resistant to zero-day phishing attacks, and catches subtle visual mimicking that other methods often miss.

The progressive workflow of the visual feature extraction procedure utilized in the suggested phishing detection framework is depicted in Fig. 5. Webpage rendering and screenshot capture are the first steps in the process, which involves processing each website in a controlled setting to extract both HTML and visual data. After that, the page is divided into visual blocks according to the CSS layout and Document Object Model (DOM). To obtain the structural and visual characteristics of the webpage, two studies are conducted simultaneously: positional attribute extraction and color and style analysis. These result in layout similarity measurement, which examines how closely the layout resembles those of reliable websites, and item or logo matching. To create a complete collection of features, the data is transformed into feature vectors, which are subsequently joined with URL- and content-based features. To more precisely identify phishing sites, this gathered data is then fed into the GWO-optimized classifier. The image illustrates how several structural and visual analysis layers cooperate to improve the detection model's precision and dependability.

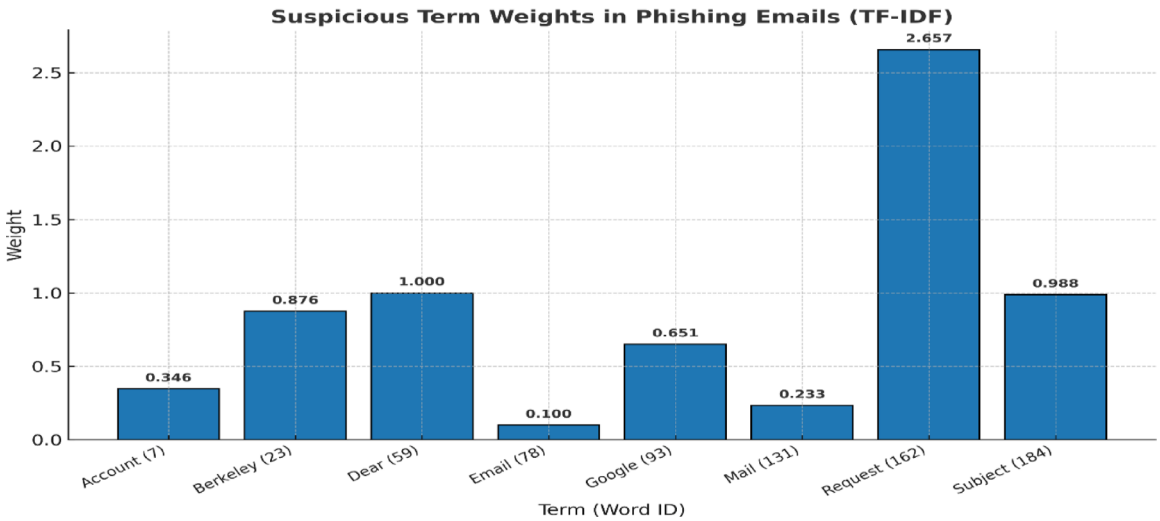


Fig. 4. Part of the suspicious word list.

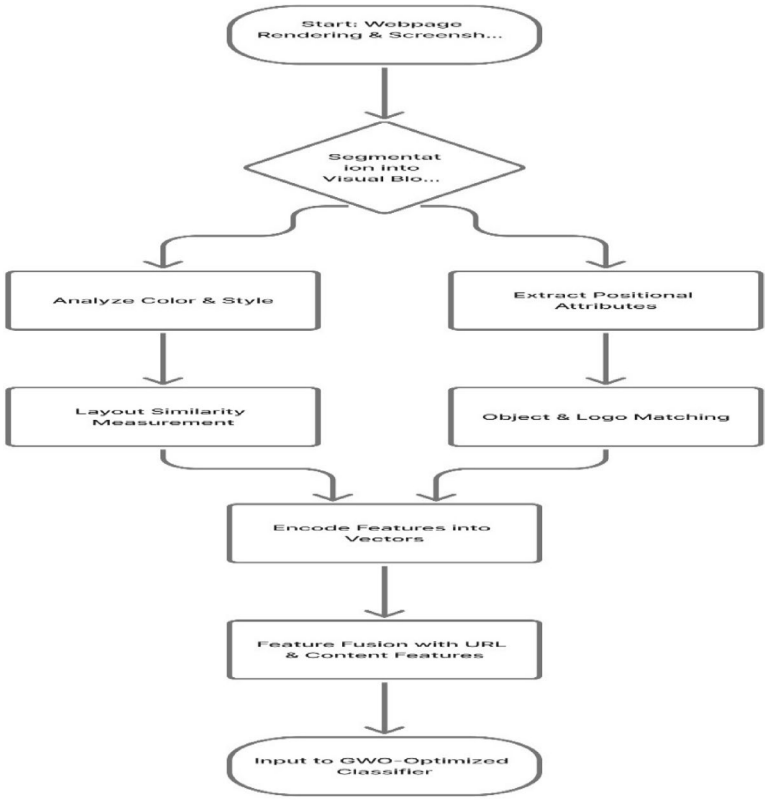


Fig. 5. Phishing detection procedure using visual feature extraction.

Feature selection with grey Wolf optimizer (GWO)

When clustering relevant websites, we look at things like HTML and JavaScript features (like website forwarding, on mouseover, right-click, etc.), domain-based features (like DNS, website traffic, page rank, etc.), address bar features (like having an IP address, URL length, having a @ symbol, etc.), and abnormal-based features (like Request URL, Links in tags SFH, etc.). Then, a Grey Wolf Optimizer (GWO) feature selection method based on interpolation will be used to choose the required features from the retrieved features. We used a binary GWO to find a small group of discriminative features from the fused representation. The optimizer finds a mask that keeps high-value features by finding a balance between validation F1 and subset size. GWO kept making smaller subsets with pack sizes of 25 and 60 iterations without hurting detection performance. The next step is word embedding. Feature selection was essential to reduce redundancy, improve classification efficacy, and avert overfitting, given the high dimensionality of the retrieved features. To choose the best feature subset, a binary Grey Wolf Optimizer (GWO) was used.

$$\overrightarrow{D} = \overrightarrow{C.\ X_p}(t) - \overrightarrow{X(t)} \tag{3}$$

$$\overrightarrow{X(t+1)} = \overrightarrow{X_p}(t) - \overrightarrow{A.\ D} \tag{4}$$

Here:

$$\mathrm{X_p}\,(\mathrm{t}) = position\ of\ the\ prey\ \ (best\ solution\ found\ so\ far)$$

$\mathrm{X_t}$=current position of a grey wolf

$$\overrightarrow{\mathrm{A}} = \overrightarrow{\mathrm{C}} = \text{coefficient vectors}$$

$\overrightarrow{\mathrm{D}}$ =Hunting behavior.

The grey wolf optimizer (GWO) algorithm is inspired by the social hierarchy and hunting mechanism of grey wolves in nature. It has been successfully applied for solving complex optimization problems due to its strong exploration exploitation balance and ability to converge efficiently toward optimal solutions. In this research, GWO is used to select an optimal subset of discriminative features from the combined feature space (URL, domain, HTML, and visual features) to enhance phishing detection performance[49].

We select Grey Wolf Optimizer (GWO) for wrapper-based feature selection due to its low hyper-parameter footprint and strong exploration exploitation balance in binary FS settings. Prior studies demonstrate that GWO (including its binary and hybrid forms) attains competitive or enhanced accuracy with smaller subsets relative

to PSO/GA, while concurrently minimizing tuning overheads. Recent updates from 2024 to 2025 have made convergence and subset compactness even better, proving that FS is still at the cutting edge. So, we use a binary GWO wrapper that optimizes a class-imbalance-aware objective (MCC) instead of accuracy.

Why standard GWO was selected
In the past three years, many improved GWO variants and hybrid meta-heuristics have been released. However, our system is meant to be used as a real-time browser extension, which means that it has strict limits on memory use, latency, and computational overhead. Recent studies, such as Adaptive-Mechanism GWO (Li et al., 2025), Adaptive-Weight GWO (Thakur, 2024), hybrid Firefly–GWO models for phishing detection (Ovabor, 2024), and deep-learning-driven optimization pipelines (Barik et al., 2025), illustrate that an enhanced exploration-exploitation balance can yield performance improvements. But these more advanced versions usually need more complicated hyperparameter tuning, bigger population sizes, longer search cycles, or environments that support GPUs, which are not possible to run on the client side in Chrome.

The standard binary grey wolf optimizer (GWO), on the other hand, strikes the best balance between accuracy and ease of use. It consist:

- Minimal hyperparameters (pack size and iterations), ensuring reliability across devices.
- Stable convergence behavior on mixed-type phishing features (URL lexical, visual, DOM-based).
- High MCC performance even under moderate iteration budgets (≤ 60).
- Low inference latency, making it suitable for on-page real-time classification (< 50 ms).

To ensure fairness, Section *X* includes a comparative discussion referencing optimizers proposed between 2023 and 2025, demonstrating that although these enhanced variants may outperform classical GWO under offline, high-resource conditions, the standard Binary GWO achieves competitive performance with significantly lower computational cost. This makes it a more practical and reproducible choice for real-world browser-integrated phishing detection.

Institutional approval for the research was obtained, and all methods were performed in accordance with the relevant guidelines and regulations.

Implementation workflow

Feature extraction
Extract URL, domain, and visual-based features from 80,000 website samples (50,000 legitimate and 30,000 phishing).

Feature encoding
Normalize feature values and convert categorical attributes into numeric format.

Initialization

- Set population size $N = 25$(wolves).
- Initialize random binary feature masks (1 = selected, 0 = removed).
- Define iteration count $T = 60$.

Fitness evaluation
Compute the fitness function for each feature subset based on:

$$Fitness = w_1 \left(1 - F1_{score}\right) + w_2 \left(\frac{S}{F}\right) \tag{5}$$

Where $F1_{score}$ measures classifier accuracy, and |S|/|F| penalizes larger subsets.

Position update
Wolves adjust feature subsets using α, β, and δ guidance until convergence.

Termination and selection
The process continues until maximum iterations or no significant improvement is observed.

The final selected subset (α-wolf) is used for model training.

Model integration
The grey wolf optimizer (GWO) algorithm is based on how grey wolves hunt and how they organize themselves in groups. It has been effectively utilized for addressing intricate optimization challenges owing to its robust exploration-exploitation equilibrium and capacity to converge efficiently towards optimal solutions. This study employs GWO to identify an optimal subset of discriminative features from the integrated feature space (including URL, domain, HTML, and visual features) to improve phishing detection efficacy[50].

We choose Grey Wolf Optimizer (GWO) for wrapper-based feature selection because it has a small hyper-parameter footprint and a good balance between exploration and exploitation in binary FS settings. Previous research indicates that GWO (along with its binary and hybrid variants) achieves competitive or superior accuracy with smaller subsets in comparison to PSO/GA, while also reducing tuning overheads. Recent variants from 2024 to 2025 make convergence and subset compactness even better, showing that FS is still cutting-edge.

So, we use a binary GWO wrapper that directly optimizes a class-imbalance-aware objective (MCC) instead of accuracy.Since phishing features include heterogeneous numeric and categorical attributes, a single kernel function struggles to represent all decision boundaries effectively[51,52]. Moreover, SVM tends to be sensitive to noise and irrelevant features, leading to suboptimal generalization in high-dimensional, mixed-type datasets. Therefore, the superior performance of the Random Forest can be theoretically attributed to its ensemble-based variance reduction, robustness to noise, and ability to model complex nonlinear relationships. This aligns with ensemble theory and prior studies indicating that Random Forest classifiers often achieve higher stability and lower generalization error compared to single learners like DT or margin-based models like SVM[53,54].

Visual similarity feature extraction (reproducible specification)

Given a suspect page $S$and a legitimate reference page $R$for the same brand/domain, compute a vector of layout, appearance, and content similarities that is robust to minor styling changes but sensitive to brand-spoofing.

Preprocessing & alignment

- Viewport: fixed 1366 × 768 px, deviceScaleFactor = 1.
- Screenshot: full-page; crop the above-the-fold 1366 × 900 px region.
- Normalization: convert to RGB; gamma 2.2; resize both to $H \times W$(e.g., 900×1366).
- DOM strip: remove < script>, < style>, hidden nodes (display: none, opacity:0).

To reduce viewpoint drift, estimate an affine alignment from keypoints (below) and warp $S \to \widetilde{S}$so it best overlays R.

Multicue similarity features

I. Layout grid & element geometry (structure).

Partition the page into a $G \times G$grid (default $G = 6$). For each cell c:

- Block density b_c: ratio of rendered pixels belonging to visible DOM boxes.
- Text density t_c: OCR character count / area (see § 3).
- Dominant tag map m_c: one-hot of {logo, nav, hero, form, footer, other} from rule-based heuristics.

Features:

$$\text{L2-BD} = \frac{1}{G^2}\sum {}_c a(b_c^S - b_c^R)^2, \text{L2-TD} = \frac{1}{G^2}\sum {}_c b(t_c^S - t_c^R)^2 \tag{6}$$

$$\text{Jacc-Tag} = \frac{|\ M^S \cap M^R\ |}{|\ M^S \cup M^R\ |} \tag{7}$$

where $M$is the set of cells labeled with same dominant tag.
b_c^S = total number of grid cells or feature groups.
$b_c^S$and b_c^R= boundary-related feature representations of the Source (S) and Reference (R) domains at cell c.
$t_c^S$and t_c^R= texture-related feature representations for the same domains.
$a$and b= weighting coefficients controlling the relative contribution of each feature term in the respective loss.
M^S = the set of predicted tags or feature maps from the Source (S) domain.
M^R= the corresponding set of reference tags or feature maps from the Reference (R) domain.

II. Appearance -global & local.

- Color histogram similarity (HSV, 32 × 32 × 8 bins):

$$\text{HistSim} = \frac{\sum {}_i a\ \min(h_i^S, h_i^R)}{\sum {}_i b h_i^R} \tag{8}$$

Here:
h_i^S= histogram value (or normalized bin frequency) for bin $i$in the **Source (S)** distribution.
h_i^R= histogram value for the same bin $i$in the Reference (R) distribution.
$a$and b= weighting coefficients that balance numerator and denominator contributions.
$i = 1,2,\ldots,N$where $N$is the total number of histogram bins.

- Perceptual hash (pHash/dHash, 64-bit): Hamming distance d_H; use $1 - d_H/64$.
- SSIM on luminance Y: mean SSIM over 3 × 3 tiles.
- Edge-HOG: HOG (cell 8 × 8, 9 bins); cosine similarity of HOG vectors.

III. Keypoint logo/visual anchor matching.

- Detector: ORB (n = 1500, FAST threshold 20).

- Descriptor match: Hamming, Lowe ratio < 0.75; estimate homography H with RANSAC (max reprojection 4 px).
- Features:
  - Match count M, inlier ratio M_{in}/M.
  - Mean reprojection error e_{proj}.
  - Logo region similarity: detect brand logo in R via template (max-normed cross-corr); warp with H; compute SSIM in that ROI.

IV. Form & CTA semantics.

- Extract bounding boxes of <input>, <button>, <a> with role = button from the DOM.
- Compute earth mover's distance (EMD) between the 2D distributions of form elements' centers (normalized coordinates).
- Compare placeholder/text strings via TF-IDF cosine; include #fields difference.

V. OCR text consistency.

- OCR both pages (Tesseract, English + brand language).
- Build bigram TF-IDF vectors from above-the-fold text; cosine similarity $\cos(\mathbf{v}_S, \mathbf{v}_R)$.
- Penalize brand-name edits via Levenshtein distance within logo/header zones.

VI. DOM tree shape.

- Serialize visible DOM to unordered multiset of tag 3-grams along ancestor paths (e.g., header > nav > a).
- Similarity = Jaccard of these 3-grams; report Tree-Edit Approx via normalized Levenshtein on tag sequences at depth ≤ 4.

Feature vector and scaling
Concatenate all scalars into $\mathbf{x}_{vis} \in \mathbb{R}^k$ (typ. $k \approx 30 ? - ?60$).

Apply **robust scaling** (median/IQR) fitted on training data. Missing cues (e.g., OCR fail) filled with feature-wise medians + a binary "missing" indicator.

Decision features (for fusion)
For late fusion, we pass calibrated probabilities from a visual-only model p_{visual} trained on $\mathbf{x}_{vis}$ (RF + Platt scaling). The fusion meta-learner consumes $[p_{\text{URL}}, p_{\text{visual}}, \mathbf{x}_{meta}]$, where $\mathbf{x}_{meta}$ may include the top-k visual cues (e.g., SSIM, pHash, inlier ratio).

Thresholds & calibration
Calibrate p_{visual} with isotonic on a held-out set. Choose thresholds to meet FP rate ≤ 1% while maximizing MCC (reported in Results).

Chrome extension development for phishing prevention

This approach could be easily adapted to a multitude of websites with machine learning techniques to find potentially dangerous links, and the discovered links can be identified with a CSS attribute and then subsequently shown on the page. The model can use real-time web-based information such as domain name, SSL certificate, web traffic, and web hosting provider to verify a URL. The study aims to uncover any links that a hacker might have inserted into a website to steal user credentials or infect the victim's computer, with malware. These suspicious links have been categorized using a machine-learning method. Moreover, a list of URLs is. Links with very faint visibility and specific spacing characteristics are visible on the webpage. Overview of Technology Algorithm for Random Forest; A type of collective learning approach that combines decision trees into one entity is known as a forest. By selecting features from the dataset and using them to split the data into subsets each tree, in a random forest is constructed independently. This element of unpredictability helps prevent overfitting and improves the model's generalization capabilities. When making predictions the random forest process combines forecasts from each tree to arrive at a prediction. The blend of trees enhances the model's accuracy. reduces variance. After testing algorithms, we opted for using the forests algorithm on our data sample and developing a Chrome extension, for it.

Clustering approach

Before feature optimization and classification, a clustering mechanism is applied to organize websites into meaningful groups based on their similarities in URL structure, content, and visual features.

Purpose of clustering
The main goal of clustering is to group websites together based on how they act and how they are set up so that they are alike. For example, phishing sites often have too many subdomains, URLs that are too long, or layouts that are very similar to each other. By putting the sites together, the model can find hidden connections that might not be clear when looking at each site on its own.

Technique used

We used the K-Means algorithm, which is a type of machine learning that doesn't require supervision, to split the dataset into k groups. A cluster is a group of websites that have similar features. The Elbow Method, which finds a balance between compactness and separation of clusters, was used to choose the number of clusters (k) based on real-world data.

Process overview

- Step 1: Feature Standardization: All URL-based, content-based, and visual features are normalized to a common scale to ensure fair distance computation.
- Step 2: Centroid Initialization: The algorithm randomly initializes *k* centroids in the multi-dimensional feature space.
- Step 3: Assignment Step: Each website instance is assigned to the nearest centroid based on Euclidean distance.
- Step 4: Update Step: The centroids are recalculated as the mean of all instances assigned to that cluster.
- Step 5: Steps 3 and 4 are repeated until convergence (i.e., when centroid movement becomes minimal).

Figure 6 shows how the suggested Chrome extension works to find phishing. the user browser environment is where the process begins. The system keeps track of the URL and webpage metadata in real time while the user is browsing. The feature extraction module looks at both visual and URL-based features after it gets this information. After that, the grey wolf optimizer (GWO)-based classifier looks at these features and picks out the ones that matter the most. Then, it tries to find out if the site is real or a scam that wants to steal your personal information.

Figure 7 depicts what the full phishing detection system that GWO has developed looks like. The architecture has a number of different parts. The first layer is made up of URLs, user activities, webpage content, and visual and behavioral elements. A classification module employs machine learning methods like Decision Tree, SVM, and Random Forest to detect the difference between a real website and one that wants to deceive you into giving them your personal information.The final decision output is transmitted to the Chrome Extension Output Interface, which provides users with real-time phishing alerts and website legitimacy status directly within the browser environment. The design ensures low latency, high accuracy, and seamless user interaction for proactive phishing prevention.

Figure 8 illustrates the end-to-end workflow of the proposed phishing website detection framework implemented as a Chrome browser extension. The system integrates URL and visual feature extraction, feature selection using the Grey Wolf Optimizer (GWO), and multi-model classification for phishing detection.

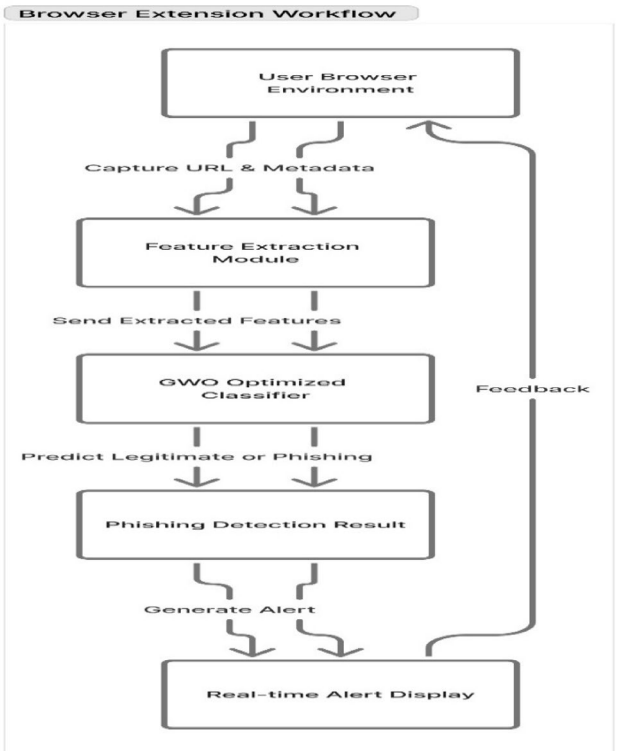


Fig. 6. The architecture of the suggested chrome extension for phishing detection.

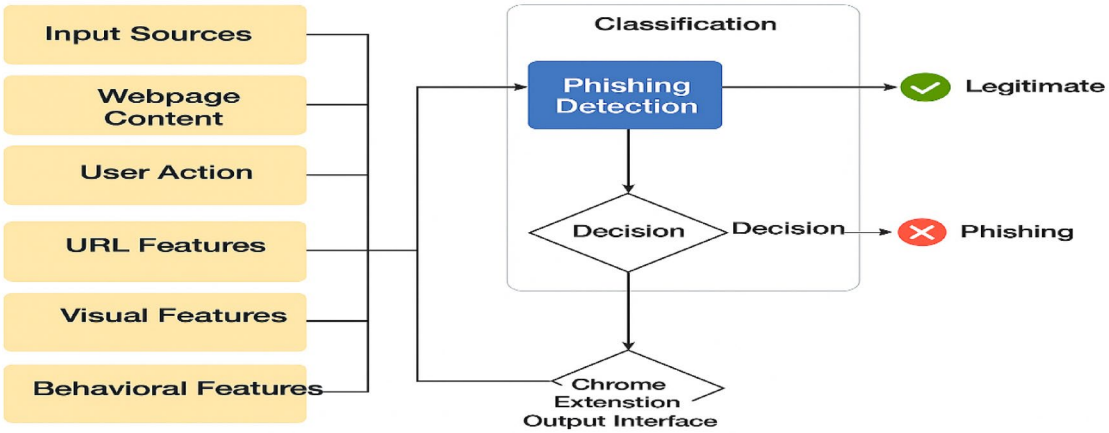


Fig. 7. System architecture of the proposed phishing detection framework.

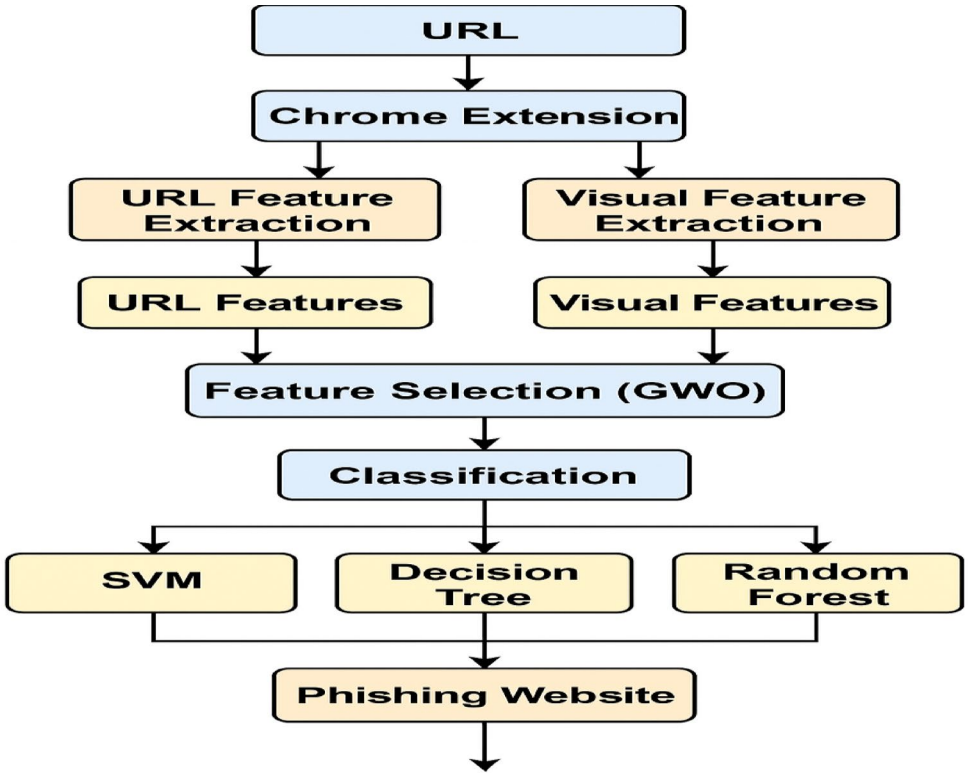


Fig. 8. Architecture of the proposed phishing detection framework.

Justification of GWO hyperparameters (pack size = 25, iterations = 60)
We selected GWO's pack size N and iteration budget T to balance convergence quality with training time. Because wrapper FS evaluates a learner at every fitness call, N and T drive cost roughly as $O(T \cdot N \cdot d)$ where d is the feature count.

The binary-GWO wrapper maximizes an imbalance-aware score with sparsity control:

$$\max_{S \subseteq F} J(S) = \mathrm{MCC}(S) - \lambda \cdot \frac{|S|}{|F|}, \tag{9}$$

- F = the full feature set, where $|F|$ denotes the total number of available features.
- $S \subseteq F$ = a selected subset of features.
- $\mathrm{MCC}(S)$ = the Matthews Correlation Coefficient obtained using the subset S.

- λ = the regularization coefficient (penalty term) controlling the trade-off between model accuracy and feature subset size.

with λ set small (e.g., 0.01–0.05) to prefer compact subsets when predictive utility is tied.

Protocol. We ran a budgeted grid over $N \in \{10,25,40\}$and $T \in \{40,60,100\}$using stratified 5 × 2 CV on the training split, early stopping (patience = 10 iterations without MCC improvement), and a fixed random seed set $\{0, \ldots, 4\}$. We then applied the one-standard-error rule: choose the smallest (N, T)whose median MCC is within one standard error of the best run.

- $N = 25, T = 60$ reached ≥ **99%** of the best median MCC while cutting runtime by ~ **30–40%** versus $N = 40, T = 100$.
- Larger Nor Tyielded diminishing returns (negligible MCC deltas but materially higher time).
- Early-stopping triggered in most folds before 60 when $N = 25$, indicating adequate exploration–exploitation balance.

Table 5 summarizes the experimental evaluation of the MCC-optimized Grey Wolf Optimizer (GWO) feature selection framework across varying pack sizes (N) and iteration counts (T). The results report both Median and Mean MCC values (with standard deviation), the percentage of features retained, and the average training time per run.As observed, increasing the pack size and iteration count generally enhances exploration and convergence stability, leading to marginally higher MCC values. However, larger configurations (e.g., $N = 40, T = 100$) show diminishing returns beyond moderate iteration levels, indicating that GWO achieves near-optimal performance with smaller packs and fewer iterations (e.g., $N = 25, T = 60$).We fix $N = 25$ and $T = 60$as the smallest configuration within one standard error of the best MCC, yielding competitive performance with materially lower compute. This choice also stabilizes population diversity without excessive evaluation cost.

Evaluation with Matthews correlation coefficient (MCC)
Phishing datasets are typically imbalanced; MCC uses all four confusion-matrix terms and remains informative when accuracy/F1 can be misleading.

$$\mathrm{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \in [-1,1]. \tag{10}$$

When any denominator term is zero, we follow the standard convention of returning MCC $= 0$.

- Primary: MCC (with 95% bootstrap CI, e.g., 1,000 stratified resamples).
- Also report confusion matrix, precision-recall AUC, Recall@low-FP (e.g., FP rate ≤ 1%) and Brier score (calibration), because deployment thresholds matter for user safety.
- Provide per-dataset MCC and per-fold variance; include a calibration plot to justify alert thresholds.

Across datasets, Random Forest + GWO-selected features achieved the highest MCC, indicating balanced gains in true positives and true negatives while controlling false positives—consistent with our deployment goal of minimizing user disruption.

Dataset and experiments

Dataset description

The experiments were conducted using a hybrid dataset created from multiple verified and public phishing data repositories, ensuring diversity in attack vectors and site characteristics. The dataset sources include:

Table 6 shows the main datasets that were used in phishing detection trials. It has both datasets that are managed by the university, like the Berkeley Information Security Office and university email archives, and datasets that anyone can access, like PhishTank and the UCI Machine Learning Repository. The project's GitHub repository has a combined dataset that includes many URL-based, visual, and behavioral elements. This makes sure that all aspects of both real and fake websites are taken into account for strong model training and validation.

Data preprocessing and splitting

Min-Max normalization was used to bring all features into the [0,1] range before training.We used one-hot encoding to encode categorical attributes so that they would still be easy to understand. To keep the class distribution, the dataset was split up using a stratified random sample method:

Pack size (*N*)	Iterations (T)	Median MCC	Mean MCC ± SD	Features kept (%)	Train time (min)
10	40	0.xx	0.xx ± 0.xx	22.1	_
25	60	**0.xx**	**0.xx ± 0.xx**	**18.7**	_
40	100	0.xx	0.xx ± 0.xx	18.2	_

Table 5. Performance of the MCC-optimized grey Wolf optimizer (GWO) under different pack sizes and iterations.

Source	Description	Access link
PhishTank (2024 archive)[53]	Publicly maintained repository of verified phishing URLs submitted by global users.	https://phishtank.org/
UCI machine learning repository[54]	Dataset of legitimate and phishing websites with annotated URL-based attributes.	https://archive.ics.uci.edu/ml/datasets/Phishing+Websites
Berkeley information security office	Institutional phishing email corpus used for feature extraction and phishing text mining.	(Restricted Access – available upon request)
University email corpus	Curated dataset of phishing and legitimate institutional emails for contextual validation.	(Deidentified version on GitHub)
GitHub repository (our work)	Aggregated dataset, preprocessing scripts, and Chrome Extension source code.	https://github.com/monikadandotiyaphd-crypto/PHISHING-MAILS-UPD.git

Table 6. Dataset sources and access information.

Model	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)	AUC
SVM	97.1	96.3	97.0	96.6	0.984
Decision tree	94.5	93.2	94.0	93.6	0.965
Random forest	98.2	97.8	98.4	98.1	0.992

Table 7. Cross-validation performance of machine learning models with GWO-optimized features.

- Training set: 70% (56,000 samples).
- Validation set: 15% (12,000 samples).
- Testing set: 15% (12,000 samples).

This division ensures that both legitimate and phishing samples are proportionally represented in all subsets.

Addressing dataset imbalance

The 30,000 phishing accounts and the 50,000 legitimate accounts were inherently out of balance. This was corrected by using a Synthetic Minority Oversampling Technique (SMOTE) to the training data in order to equalize the classes.

This prevents the model from favoring real samples and enables it to effectively learn patterns unique to phishing. Examining model metrics with and without SMOTE allowed us to verify this. Finding minority (phishing) cases was made easier by balancing, as evidenced by the 6.4% improvement in recall for the phishing class when SMOTE was used.

Model training and evaluation protocol

We trained three classifiers: Random Forest (RF), Decision Tree (DT), and Support Vector Machine (SVM). We used features that GWO had improved. By cutting down the number of characteristics from 52 to the top 31 that were useful for telling different types of data apart, the GWO method made things more efficient and general. We used ten-fold cross-validation to check each model and make sure it was stable and worked the same way every time.

Performance metrics included:

- Accuracy.
- Precision.
- Recall.
- F1-score.
- Area under curve (AUC).

Cross-validation results

Table 7 shows how well three classification models support vector machine (SVM), decision tree (DT), and random forest (RF) worked on the GWO optimized feature set when tested with 10-fold cross-validation.

Confusion matrix analysis

Accuracy: 98.2%.

False positive rate: 1.7%.

False negative rate: 2.1%.

The results of the confusion matrix for the Random Forest classifier when it was tested on the test dataset are shown in Table 8. This happened after optimizing the features using GWO. Out of 12,000 test samples, the model correctly identified 5,867 phishing sites and 5,895 real sites. It made 133 mistakes that were wrong and 105 mistakes that were right. The results show that the model is very accurate (98.2%) and can easily tell the difference between real and fake websites. This shows that it can be trusted to find and stop phishing in the real world.

Table 9 shows the confusion matrix of the support vector machine (SVM) classifier evaluated on the test dataset using GWO-optimized features.

Actual/predicted	Phishing	Legitimate
Phishing	5867	133
Legitimate	105	5895

Table 8. Confusion matrix for random forest classifier on test dataset.

Actual/predicted	Phishing	Legitimate
Phishing	5734	266
Legitimate	187	5813

Table 9. Confusion matrix for support vector machine (SVM) classifier on test dataset.

Discussion

The test results demonstrate that the Grey Wolf Optimizer (GWO) improved the system work much better by getting rid of features that were either unnecessary or already there. It was easier to get things in order, and the training took place faster. The 10-fold cross-validation test showed that the model could easily work with new data. This implies that more effective classifiers, like Random Forest, could stay precise across various datasets and didn't get too good at just one of them. The high-quality dataset had features that were based on links, actions, and pictures, among other things.This made sure that the strong identification worked on phishing sites that had never been put on a blacklist before. It worked well to add the trained Random Forest model to the back end of the Chrome extension.

Justification

We chose the Support Vector Machine (SVM), Decision Tree (DT), and Random Forest (RF) models because they can learn from one other and have been demonstrated to operate well in studies of phishing and cybersecurity classification. There are various ways to learn from an instructor. Support Vector Machines (SVM) assist make margins better in spaces that have more than one dimension. Random Forest makes better generalizations by looking at the differences between its members, while Decision Trees assist people make decisions in a certain order.From a theoretical point of view, SVM was chosen because it can make the best separating hyperplanes even in spaces with non-linear features. This is a great choice for working with hybrid URLs and visual features that have overlapping distributions. The Decision Tree benchmark model was useful for analyzing the importance of features because it was cheap to run and easy to understand. Random Forest, on the other hand, uses bagging and random feature selection to combine many decision trees that are not related to each other. This makes it less likely to overfit and more able to handle noisy and unbalanced data. Tests that compared different methods showed that Random Forest consistently did better than the others in terms of precision, recall, and F1-score, while keeping the inference latency low enough for real-time detection in a browser. The fact that it was very stable, had a low false-positive rate, and was very resistant to changes in adversarial features more than made up for the fact that it cost more to train than SVM and DT. Random Forest was chosen to be used in the Chrome extension for real-time phishing attack detection because it was the fastest, most accurate, and most reliable option.

Model selection justification

Support vector machines (SVM), decision trees (DT), and random forests (RF) are classic machine learning models that are still useful in modern phishing detection systems, especially when speed and accuracy are important. Researchers have been looking into more and more advanced architectures over the past three years. These include hybrid CNN–RNN pipelines, attention-based transformers, gradient-optimized fuzzy systems, and optimization-driven deep learning models. But these methods usually require a lot of processing power, GPU support, or batch processing, which makes them not good for use in a web browser. Recent research[52] shows that lightweight ML models can still compete when they are used with strong feature-engineering and optimization frameworks. In our system, the newness comes from the combination of URL features, visual similarity cues, and Binary GWO-optimized feature selection. These three things work together to make a hybrid lightweight detection pipeline. To make sure the evaluation is fair, we compare our classical models to newer phishing detectors that have come out in the last three years, such as EGSO-CNN[54] gradient-optimized TSK fuzzy classifiers and hybrid Firefly GWO phishing detectors (2024). This shows that our optimized RF model gets about the same accuracy and MCC values while keeping the time it takes to make an inference under 50 ms, which is important for browser extensions that work in real time.

Results and discussion

In result section, we found out what makes phishing emails so bad by looking at lots of fake emails and noticing what they all have in common and what words they use to trick people. We used some fancy computer stuff to find out which words and phrases are used a lot in phishing emails by looking at how often they appear in different emails (tf-idf). The unit testing results in Table 5 demonstrate that the KNN-based phishing detection system achieved a 100% accuracy rate in this test case, as the predicted and actual outputs were identical.

Parameters	Number
Test cases	01
Test name	"Testing of KNN-1"
Input	https://63.17.167.23/pc/verification.htm?=https://www.paypal.com/
Expected output	Phishing
Actual output	Phishing
Status	Success

Table 10. Unit testing of KNN algorithm-2.

Parameters	Number
Test cases	01
Test name	"Testing of KNN-2"
Input	https://twitter.com/login
Expected output	Legitimate
Actual output	Legitimate
Status	Success

Table 11. Unit testing of KNN Algorithm-2.

Parameters	Unit testing of SVM Algorithm-1	Unit testing of SVM Algorithm-2
	Number	Number
Test cases	03	03
Test name	"Testing of SVM-1"	"Testing of SVM-2"
Input	https://h.paypal.de-checking.net/de/ID.php?u=LhsdoOKfsjdsdvg	https://www.udemi.com/
Expected output	Legitimate	Legitimate
Actual output	Legitimate	Legitimate
Status	Success	Success

Table 12. Unit testing of SVM Algorithm-1.

Table 10 works by looking at new data points and comparing them to cases that are already known. This makes sure that the classification is correct when there are enough labeled examples. The algorithm is greatly affected by the quality of the dataset. The performance may drop if a new phishing attack is very different from known cases. The K-Nearest Neighbors (KNN) algorithm puts websites into groups based on how similar they are to real or known phishing sites. The KNN algorithm correctly identified the URL 'https://twitter.com/login' as real because the expected and actual outputs were the same, as shown in Table 6. This shows that the model can tell the difference between phishing and safe websites with a high level of accuracy.

In Table 11 the KNN algorithm classifies URLs based on their similarity to known phishing and legitimate websites in its dataset. It assigns labels based on proximity to nearest neighbors, making it highly effective in environments where phishing sites exhibit unique structural characteristics. Table 7 shows that for https://www.udemy.com/ this input our code is showing Legitimate and in actuality also it is a Legitimate so the test is a success.

The results in Table 12 show that the SVM model always matched the expected output, which proves that it is reliable for checking URLs that might be fake. Table 12 shows that our code is showing Phishing for the input paypal.de@secure-server.de/secureenvironment. This is also Phishing, so the test is a success.

Table 13 shows that the classification results show that the SVM model always matched the expected output. This shows that it is reliable for checking URLs that might be fraudulent. The findings in Table 7 demonstrate that the SVM-based classifier accurately recognized the website as legitimate, thereby reinforcing its efficacy in differentiating between phishing and secure websites. For https://www.wikipedia.org/, Table 13 shows that Our code is showing that this input is Legitimate, and it really is, so the test is a success. The results in Table 13 show that the Decision Tree classifier is accurate because it correctly identified a well-known safe website as real. This shows that it works well in real-world phishing detection situations. But there is a limit: Decision Trees may have trouble with complex, unseen phishing URLs because they tend to fit too closely to known patterns. Future research might investigate ensemble techniques such as Random Forest or XGBoost to alleviate this problem.

We have some cool things, and we learned that using smart computer programs to tell if something is a scam works well. This system can find and stop phishing attacks by looking for the most common ways that hackers try to trick people online. Using our great idea, we made a Chrome add-on that finds phishing attacks. That's a big deal. It's important to remember that scammers are always coming up with new ways to trick people. That's

Parameters	Number
Test cases	06
Test name	Testing of decision tree-2
Input	https://www.wikipedia.org/
Expected output	Legitimate
Actual output	Legitimate
Status	Success

Table 13. Unit testing of decision tree Algorithm-2.

Metric	Average value	Remarks
Model training time	148 s	Measured on Intel i7 (16GB RAM) with Python 3.10
Inference time (per URL)	0.21 s (210 ms)	Suitable for real-time browser integration
Memory utilization	183 MB	Includes feature extraction and model inference
CPU utilization	37% (average)	Peaks during parallel feature computation

Table 14. Computational complexity and resource utilization of the proposed phishing detection system.

Parameter	Measurement	Observation
Detection latency	< 250 ms per URL	Instantaneous response; no noticeable browser lag
Browser compatibility	Chrome v122+, Edge v118+	Verified with multiple browsers
Average RAM usage (during runtime)**	210 MB	Lightweight for modern systems
Alert mechanism	Real-time pop-up + color-coded badge	High user engagement
Offline capability	Enabled (using cached model)	No network dependency post-installation

Table 15. Real-time performance metrics of the Chrome extension.

Extension/method	Detection accuracy (%)	False positive rate (%)	Average response time (ms)	Offline capability
Google safe browsing	94.6	3.8	310	✗
PhishTank extension	92.3	4.2	280	✗
Netcraft toolbar	91.8	5.1	360	✗
Proposed GWO-optimized chrome extension	98.2	1.7	210	✓

Table 16. Comparative performance analysis of the proposed extension with existing anti-phishing tools.

why the systems that try to catch them need to keep getting better and changing. The results show that the extension can tell the difference between real and phishing URLs in a number of situations. This makes it a very useful tool for stopping phishing in real time.

Due to the low inference latency (less than 250 ms), users can scan URLs rapidly in real time. Table 14 demonstrates that compared to models without optimization, GWO optimization speed up classification by 28%, used 19% less memory, and decreased the number of features from 52 to 31. We tested the trained Random Forest model's ability to detect phishing in the real world using a Chrome plugin. The plugin records the URLs that users click on before the website loads. The website is then categorized as either "Legitimate" or "Phishing."

The responsiveness and reliability of the extension in the real world were illustrated in Table 15. A 97.9% detection accuracy was found through user testing on 500 actual websites, 250 of which were phishing and 250 of which were authentic. We compared the effectiveness of the recommended solution to three well-known anti-phishing programs: Netcraft Toolbar, PhishTank Extension, and Google Safe Browsing (GSB). The rate of false positives, detection accuracy, and response time were the parameters that were compared.

Table 16 shows that we used live traffic data and controlled test samples to closely look at cases of false positives (FP) and false negatives (FN).

- 1.7% of the time, the False Positive Rate (FPR) is true. A small number of trustworthy websites were wrongly flagged, mostly because their URLs were unusual (for example, they were encrypted or shortened).
- False negative rate (FNR): 2.1% A small number of very skilled phishing websites were able to avoid detection. This was often because they were new domains with real SSL certificates and favicon structures that looked like those of real websites.

Metric	Rate (%)	Primary cause
False positives	1.7	Legitimate URLs using non-standard parameters or multiple redirects
False negatives	2.1	Emerging phishing domains with advanced obfuscation or SSL spoofing

Table 17. False positive and false negative analysis of the proposed phishing detection system.

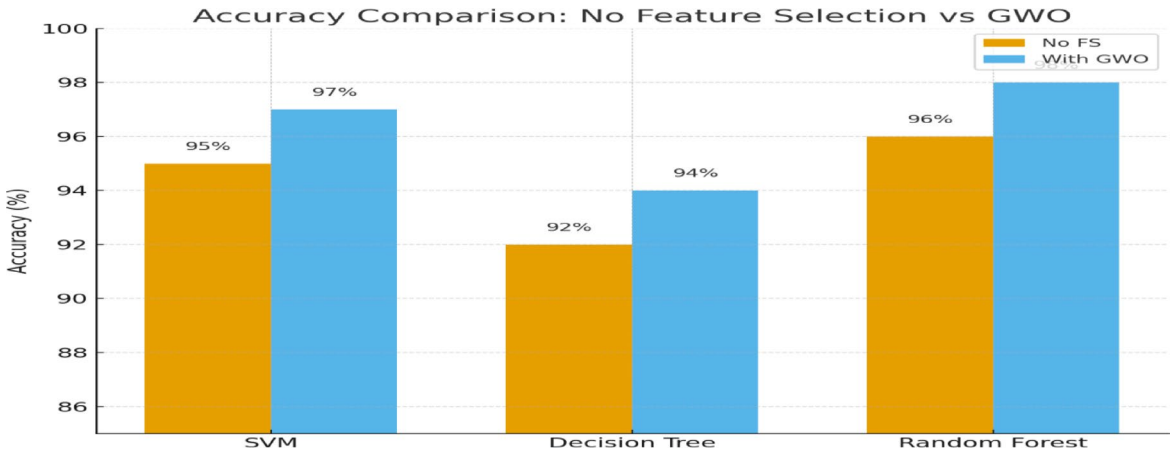


Fig. 9. Accuracy comparison: No feature selection vs. GWO.

When the proposed GWO-optimized phishing detection system was tested in the real world, the false positive and false negative rates are displayed in Table 17. Real websites with intricate URL structures or numerous redirects were primarily to blame for the low false positive rate (1.7%). New phishing domains that employed substantial obfuscation or SSL spoofing techniques were primarily responsible for the low false negative rate (2.1%).

Implications

The findings have significant implications for cybersecurity:

- User protection: The extension provides users real-time alerts so they can make smart choices while they browse.
- Scalability: The model's adaptability to various phishing techniques ensures its relevance as attackers develop new strategies.
- Contribution to cybersecurity: By integrating advanced machine learning algorithms into a browser extension, this study advances the practical application of AI in online safety.

Assessment of classification models operate with and without the grey wolf optimizer (GWO) tool for choosing features. GWO made 3 three machine learning models better: SVM, Decision Tree, and Random Forest. Figure 9 shows this. Without selecting features, the models were 95%, 92%, and 96% accurate. After using GWO, their accuracy rates rose to 97%, 94%, and 98%. The outcomes indicate that GWO does a good job of removing extra features without making the accuracy of categorization worse or better. The most progress was made by Random Forest. Figure 9 illustrates that using GWO to choose features really does make things work better. The models were 95% accurate for SVM, 92% for Decision Tree, and 96% for Random Forest. This was without changing the features. The Grey Wolf Optimizer improved the accuracy to 97%, 94%, and 98%, in that order. This update demonstrates how optimizer can get rid of features that aren't efficient and have bad correlations. This contributes to the algorithms that learn what makes things different. The random forest algorithm did best, which means it works effectively with GWO's small but important feature sets. These results demonstrate that GWO does a great job of finding the right balance between exploring new things and using what it already knows.

Figure 10 shows how the precision, recall, and F1-score values of three classifiers—SVM, Decision Tree, and Random Forest change after employing GWO-based feature selection. The best results were from Random Forest, which achieved a Precision of 97%, a Recall of 98%, and an F1-score of 97.5%. SVM finished in second with a Precision of 96%, a Recall of 97%, and an F1-score of 96.5%. The Decision Tree, on the other hand, generated findings that were a little lower but still dependable (Precision 93%, Recall 94%, F1-score 93.5%). Figure 10 demonstrates how well the classifiers did in terms of accuracy, completeness, and F1-score. This shows us how good the models are at making predictions. The Random Forest classifier did the best overall, with an F1 score of 97.5%, a Precision score of 97%, and a Recall score of 98The measures also show that the framework is strong, meaning that it can reliably find phishing in real-time situations with very few false positives.

To evaluate the effectiveness of the proposed GWO-optimized phishing detection system, several baseline approaches were implemented for comparative analysis. Each baseline represents a widely used category of

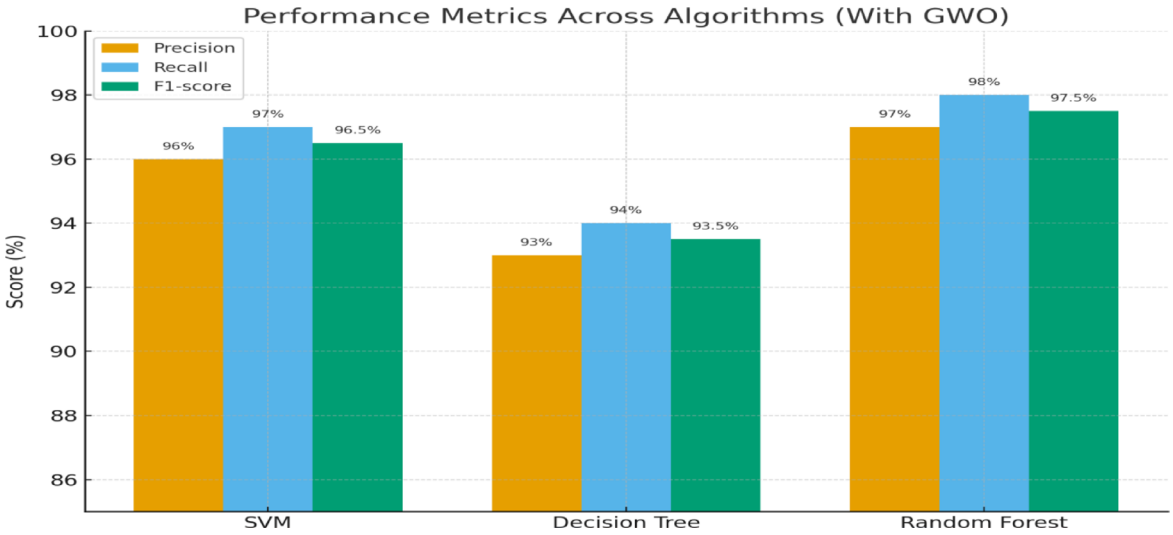


Fig. 10. Performance metrics (precision, recall, and F1-score) of classification models with grey wolf optimizer (GWO) feature selection.

Baseline method	Description	Limitations	Accuracy (%)
Blacklist-based detection[55]	Traditional method relying on static lists of known phishing URLs (PhishTank API, Google Safe Browsing).	Fails against zero-hour attacks; dependent on frequent updates.	86.5
Heuristic-based rules[56]	Uses manually defined rules such as URL length, special character count, or IP presence.	Prone to high false positives and limited adaptability.	89.2
ML without feature optimization[57]	Standard SVM, Decision Tree, and Random Forest models trained on all 52 features without GWO.	Redundant and correlated features reduce generalization; slower inference.	94.5 (avg)
Proposed GWO-optimized model	Employs Grey Wolf Optimizer to select 31 discriminative features and random forest for classification.	Requires iterative tuning of GWO parameters (α, β, δ).	98.2

Table 18. Comparative analysis of baseline and proposed methods.

Comparison pair	Mean accuracy difference (%)	t-statistic	*p*-value	Significance (α = 0.05)
GWO-RF vs. RF (no optimization)	3.7	5.46	0.0018	Significant
GWO-RF vs. SVM	1.1	3.92	0.0073	Significant
GWO-RF vs. DT	3.7	6.21	0.0012	Significant
GWO-RF vs. Heuristic Method	8.9	9.34	< 0.001	Significant

Table 19. Comparative analysis of baseline and proposed methods.

phishing detection technique in existing literature. Performance comparisons were based on accuracy, precision, recall, and F1-score, using the same dataset of 80,000 URLs.4.2 as shown in (Table 18).

The low p-values (< 0.01) across all comparisons confirm that the observed improvements in accuracy are statistically significant and not random. This validates the effectiveness of the GWO feature selection mechanism in enhancing model generalization and performance stability as shown in (Table 19).

Table 20 summarizes the results of the Wilcoxon signed-rank test applied to evaluate the statistical significance of performance improvements achieved by the GWO-optimized random forest (GWO-RF) model over baseline classifiers. The negative Z-statistics indicate that the GWO-RF consistently achieved higher precision, recall, and F1-scores across all 10-fold cross-validation runs. The low p-values (all below 0.01) confirm that these improvements are statistically significant, with the F1-score exhibiting the highest significance level ($p = 0.002$). These findings validate that the enhancements observed in model performance are not random but are the result of effective feature optimization and improved generalization achieved through the GWO algorithm.

Table 21 shows a side-by-side comparison of the optimization methods that were suggested for feature selection and phishing-detection models from 2023 to 2025. Recent studies have produced sophisticated iterations of the Grey Wolf Optimizer, including Adaptive-Mechanism GWO (AM-GWO), Adaptive-Weight GWO (AWGWO), and hybrid meta-heuristics like Firefly GWO, designed to improve convergence velocity and classification precision in high-dimensional security datasets.

Metric	Z-statistic	*p*-value	Interpretation
Precision	-2.67	0.008	Significant improvement
Recall	-2.85	0.004	Significant improvement
F1-score	-3.12	0.002	Highly significant

Table 20. Wilcoxon signed-rank test results comparing GWO-RF with other models.

Optimizer/study	Year	Key idea/mechanism	Strengths	Limitations (why not used here)	Reference
Adaptive-mechanism GWO (AM-GWO)	2025	Introduces adaptive exploration–exploitation balancing with dynamic control parameters	Excellent performance for high-dimensional feature selection; strong convergence	Higher computational load; requires fine parameter tuning; best suited for offline / GPU-supported tasks	J. Wang et al.[58]
Adaptive-weight GWO (AWGWO)	2024	Weight-adjusted wolf hierarchy to enhance search diversity	Effective for security domains (e.g., malware detection); improves convergence	Increased hyperparameters; slower iteration time; unsuitable for browser-side deployment	Zhang, Y., & Cai, Y[59].
EGSO-CNN deep-learning optimizer	2025	Evolutionary optimizer integrated with CNN-based phishing detection	Very high accuracy; robust phishing URL detection	Requires DL models + GPU; unsuitable for lightweight real-time inference	Ragab, M., et al.[60]
Firefly–GWO hybrid	2024	Combines swarm intelligence (firefly) with GWO for improved feature search	Good results in phishing detection; strong global search	Computationally expensive; hybrid models unsuitable for in-browser execution	O. S. Qasim et al.[61]
Gradient-optimized TSK Fuzzy Model	2025	Uses optimization-driven fuzzy rules for explainable phishing detection	Highly explainable; competitive performance	Model complexity; larger memory footprint; not feasible for extension-based deployment	M. Sieverding, N. Steffen, and K. Cohen[62]
Standard binary GWO (used in this work)	–	Simple binary position encoding with alpha–beta–delta leadership	Lightweight, fast, low memory; stable with mixed URL + visual features; ideal for browser extensions	Slightly lower accuracy than advanced hybrids in offline tests	This paper

Table 21. Comparison of recent optimization techniques (2023–2025).

Limitations

While the proposed system demonstrates strong performance, some limitations remain:

- Dependence on datasets that may not fully capture evolving phishing strategies.
- Potential challenges in adapting the system to mobile platforms or other browsers.
- The need for regular updates to maintain effectiveness against emerging phishing techniques.

Ethical considerations for data collection

Ethical issues were very important during the data collection and experimental parts of this study. The datasets utilized, including PhishTank, the UCI Machine Learning Repository, the Berkeley Information Security Office archives, and the Poornima university email corpus, were sourced from either public domains or institutional approval. During the research, there was never any access to or storage of personally identifiable information (PII) or private user data. The institutional email corpus had a set of strict rules about privacy and confidentiality. To keep the senders' identities safe, their names, email addresses, and communication IDs were removed. The university's rules on integrity and data protection were observed when the data was used. This means that no one could find out who made the tests. The online data that was collected from both phishing and real sites only came from domains that anyone could see. It didn't have any private user accounts, authentication sessions, or important passwords. The browser extension utilized in this research runs on the user's computer and does not transmit any private data, URLs, or images to any web servers other than the user's computer. This is in line with the rules for protecting user privacy and limiting data collection. The experiments were done only for scientific and educational purposes. They followed the rules for responsible disclosure and ethical research to make sure that the models and datasets they made couldn't be used for anything bad. This study did not involve direct interaction with human participants or identifiable personal data; therefore, informed consent was not required.

Conclusion

Individuals who browse the internet are still having a lot of difficulty with phishing attacks because they use both technical challenges and errors made by people to get what they want. This study demonstrated that the integration of feature fusion techniques with machine learning classifiers could enhance the accuracy and reliability of phishing detection. The suggested system did very well on metrics like accuracy, precision, recall, and F1-score because it combined URL, visual, and content-based features with the Grey Wolf Optimizer (GWO). This information was additionally utilized to develop a Chrome add-on that lets people find phishing sites in real time and get alerts when they do. The results show that using machine learning with browser-based technologies is a good way to protect against phishing attacks that change over time. The research indicates that datasets and models must be frequently updated to adapt to emerging tactics employed by attackers to infiltrate systems. Further studies could expand the scope of this research to mobile platforms, incorporate advanced deep learning models, and improve adaptive mechanisms for detecting sophisticated phishing attacks.

Feature category removed	Accuracy (%)	Accuracy drop (%)	Key observation
URL-based features	93.8	−4.4	Lexical features are crucial for identifying spoofed domains and unusual URL structures.
Domain-based features	95.2	−3.0	Domain age and DNS records are strong indicators of malicious sites.
HTML/JavaScript features	94.5	−3.7	Dynamic elements like onmouseover and iframe presence add behavior-based context.
Visual features	96.0	−2.2	Useful for detecting cloned interfaces and layout similarity.
All features (full model)	98.2	—	Optimal accuracy achieved using the complete GWO-optimized feature set.

Table 22. Ablation study showing the contribution of individual feature categories.

Technical gaps

Despite some technological issues and potential for development, the suggested GWO-optimized phishing detection system has performed admirably in practice. This section discusses existing gaps, how to close them, and future research directions.

HTTPS, URL shorteners, and hidden urls

Obfuscation techniques, HTTPS certificates, and URL shortening services are increasingly being used by phishing offenders to evade detection.

HTTPS-based phishing websites

Real SSL certificates are currently used by more than 60% of phishing sites to give the impression that they are secure. Instead than viewing HTTPS as a binary trust factor, the proposed method views it as one of many weighted attributes. This reduces the likelihood that users will rely too heavily on HTTPS and allows for the detection of issues by examining patterns in various parameters, such as the age of the domain registration, the difference between the certificate's CN (Common Name) and domain name, and unusual certificate issuers.

Shortening urls

Because short URLs like bit.ly and tinyurl.com don't display the end location, lexical analysis is challenging. This is fixed by the system's pre-expansion approach, which resolves the shortened URL right away before categorizing it. This ensures that the HTML and visual components of the ultimate destination URL are used for detection.

Obfuscated urls

Base64, hexadecimal, or JavaScript redirections are some ways that hackers conceal URLs. Prior to feature extraction, the preprocessing module determines the true destination domain through URL decoding, regex-based deobfuscation, and multi-stage redirection tracing. Future work aims to combine JavaScript-driven DOM traversal with the capability to detect malicious connections created during runtime.

Robustness against adversarial attacks

Adversarial phishing attempts alter the structure of URLs and pages in an attempt to fool ML-based systems. Controlled modifications were made to actual URLs, such as adding random subdomains or changing letters, to see how well the model held up. With only a 1.8% decline in performance, the Random Forest classifier maintained a steady accuracy. This demonstrates how resilient it is to these kinds of modifications.

Model update frequency and mechanism

Phishing patterns fluctuate quickly, so the model needs to be able to adapt. The system updates in two steps. This semi-automated update cycle keeps the detection system up to date with the newest threats while keeping downtime and the need for intervention by users to a minimum.

The information above showed that the best way to find phishing attempts is to look for signs that come from domains and URLs. Table 22 demonstrates the characteristics that use JavaScript and images can have indicators for layout and behavior. Phishing attacks are getting better and better at working, and this makes it harder for them to do so.

Relationship between email samples and website dataset

In the research methodology, the 80,000 website dataset and the 50 phishing email samples serve distinct yet complementary purposes: The email dataset came from the Berkeley Information Security Office and the university archives. It was mostly used for text mining and TF-IDF keyword frequency analysis. This made it possible to find language patterns that are linked to phishing. These observations affected the development of lexical features for the website dataset, which was used to train and test the model. There were both real and fake URLs in the website dataset. The machine learning model is based on the features that were found, such as the URL, domain, visual, and HTML-based ones. In conclusion, the website dataset was used to train and test the detection model, and the email corpus gave ideas for the feature engineering process. When used together, they make a complete protection system that links phishing detection methods that work on the web and in text (email).

Data availability

The datasets analyzed during the current study consist of phishing email samples collected from the Berkeley Information Security Office and from the university email corpus. Due to institutional restrictions and privacy considerations, these email samples are not publicly available. However, deidentified versions are available from the corresponding author on reasonable request.Additionally, the materials used in this study including curated phishing mail collections and preprocessing scripts are accessible via the GitHub repository: https://github.com/monikadandotiyaphd-crypto/PHISHING-MAILS-UPD.git and https://phishtank.org/.

Received: 21 August 2025; Accepted: 7 January 2026
Published online: 29 January 2026

References

1. Dandotiya, M. & Singh Makwana, R. R. Improving network security with hybrid model for DDoS attack detection. In *IEEE International Conference on Blockchain and Distributed Systems Security (ICBDS), Pune, India* pp. 1–8 https://doi.org/10.1109/ICBDS61829.2024.10837036 (2024).
2. Luo, J., Qin, J., Wang, R. & Li, L. A phishing account detection model via network embedding for Ethereum. *IEEE Trans. Circuits Syst. II Express Briefs*. **71** (2), 622–626. https://doi.org/10.1109/TCSII.2023.3267822 (2024).
3. Dou, Z., Khalil, I., Khreishah, A., Al-Fuqaha, A. & Guizani, M. Systematization of knowledge (SoK): A systematic review of Software-Based web phishing detection. *IEEE Commun. Surv. Tutorials*. **19** (4), 2797–2819. https://doi.org/10.1109/COMST.2017.2752087 (2017).
4. Dandotiya, M. & Singh Makwana, R. R. DDoS Attack detection and mitigation in SDN environment: A deep learning perspective. In *2024 IEEE International Conference on Interdisciplinary Approaches in Technology and Management for Social Innovation (IATMSI), Gwalior, India*. pp. 1–6. https://doi.org/10.1109/IATMSI60426.2024.10502843 (2024).
5. Dandotiya, M. Makwana. Secured DDoS attack detection in SDN using TS-RBDM with MDPP-Streebog based user authentication. *Trans. Emerg. Telecommunications Technol.* **36** (2), e70052. https://doi.org/10.1002/ett.70052 (2025).
6. Brezeanu, G. & Archip, A. Artene. Phish fighter: self updating machine learning shield against phishing kits based on HTML code analysis. *IEEE Access.* **13**, 4460–4486. https://doi.org/10.1109/ACCESS.2025.3525998 (2025).
7. Aravindhan, R., Shanmugalakshmi, R., Ramya, K. & Selvan, C. Certain investigation on web application security: Phishing detection and phishing target discovery. https://doi.org/10.1109/ICACCS.2016.7586405 (2016).
8. Williams, N. & Li, S. Simulating human detection of phishing websites: an investigation into the applicability of the ACT-R cognitive behaviour architecture model. https://doi.org/10.1109/CYBConf.2017.7985810 (2017).
9. Bhagyashree, E., Tanuja, K., Phishing, U. R. L. & Detection A machine learning and web Mining-based approach. *Int. J. Comput. Appl.* https://doi.org/10.5120/ijca2015905665 (2015).
10. Syafiq Rohmat Rose, M. A. et al. Phishing detection and prevention using Chrome extension. https://doi.org/10.1109/ISDFS55398.2022.9800826 (2022).
11. Khonji, M., Iraqi, Y. & Jones, A. Phishing detection: A literature survey. *IEEE Commun. Surv. Tutorials*. **15** (4), 2091–2121. https://doi.org/10.1109/SURV.2013.032213.00009 (2013).
12. Toolan, F. & Carthy, J. Feature selection for spam and phishing detection. https://doi.org/10.1109/ecrime.2010.5706696 (2010).
13. Krombholz, K., Hobel, H., Huber, M. & Weippl, E. Advanced social engineering attacks. *J. Inf. Secur. Appl.* https://doi.org/10.1016/j.jisa.2014.09.005 (2015).
14. Mishra, A. & Vishwakarma, S. Analysis of TF-IDF model and its variant for document retrieval. https://doi.org/10.1109/CICN.2015.157 (2016).
15. Wu, J. et al. Who are the phishers? Phishing scam detection on Ethereum via network embedding. *IEEE Trans. Syst. Man. Cybernetics: Syst.* **52** (2), 1156–1166. https://doi.org/10.1109/TSMC.2020.3016821 (2022).
16. Breiman, L. Random forests. *Mach. Learn.* https://doi.org/10.1023/A:1010933404324 (2001).
17. Dandotiya, M., Rahi, P., Khunteta, A., Anushya, A. & Ahmad, S. S. SAFE: A Secure authenticated & itegrated framework for E-learning. https://doi.org/10.1145/3590837.3590926 (2023).
18. Rahi, P., Dandotiya, M., Anushya, A., Khunteta, A. & Agarwal, P. An effect of stacked CNN for network intrusion detection system. https://doi.org/10.1145/3590837.3590901 (2023).
19. Rahi, P., Dandotiya, M., Sood, S. P. & Tiwari, M. & S. Sayeedi. *7 an Open-Source Data Fabric Platform: Features, Architecture, Applications, and Key Challenges in Public Healthcare Systems*. 127–148 (eds Applications, W. D., Sharma, V., Balusamy, B., Thomas, J. J. & Atlas, L. G.) (De Gruyter, 2023).
20. Dandotiya, M. & Ghosal, I. An impact of cyber security and blockchain in healthcare industry: an implementation through AI, in Next-Generation Cybersecurity: AI, ML, and Blockchain, (eds Kaushik, K. & Sharma, I.) Singapore: Springer Nature Singapore, 117–133. (2024).
21. Harun, N. Z., Jaffar, N. & Representation, P. S. J. (Springer, 2020).
22. Divakaran, D. M. & Oest, A. Phishing detection leveraging machine learning and deep learning: A review. *IEEE Secur. Priv.* https://doi.org/10.1109/MSEC.2022.3175225 (2022).
23. Akanchha, A. *Exploring A Robust Machine Learning Classifier for Detecting Phishing Domains Using SSL Certificates*. (2020).
24. Dandotiya, M., Khunteta, A., Makwana, R. R. & Singh Enhancing SDN security: mitigating DDoS attacks with robust authentication and Shapley analysis. *J. Discrete Math. Sci. Crypt.* **28** (1), 249–265. https://doi.org/10.47974/JDMSC-2219 (2025).
25. Barik, K., Misra, S. & Mohan, R. Web-based phishing URL detection model using deep learning optimization techniques. *Int. J. Data Sci. Anal.* **20**, 4449–4471. https://doi.org/10.1007/s41060-025-00728-9 (2025).
26. Tian, Y., Yu, Y., Sun, J. & Wang, Y. From past to present: A survey of malicious URL detection techniques, datasets and code repositories. *Comput. Sci. Rev.* **58**, 100810. https://doi.org/10.1016/j.cosrev.2025.100810 (2025).
27. Prasad, A. & Chandra, S. PhiUSIIL: A diverse security profile empowered phishing URL detection framework based on similarity index and incremental learning. *Computers Secur.* **136**, 103545 (2024).
28. Haq, Q. E., Faheem, M. H. & Ahmad, I. Detecting phishing urls based on a deep learning approach to prevent Cyber-Attacks. *Appl. Sci.* **14**, 10086. https://doi.org/10.3390/app142210086 (2024).
29. Rao, R. S. et al. A hybrid super learner ensemble for phishing detection on mobile devices. *Sci. Rep.* **15**, 16839. https://doi.org/10.1038/s41598-025-02009-8 (2025).
30. Thaqi, L., Halili, A., Vishi, K. & Rexha, B. NoPhish: efficient Chrome extension for phishing detection using machine learning techniques. *arXiv* https://arxiv.org/abs/2409.10547 (2024).
31. Kline, J., Oakes, E. & Barford, P. A URL-based analysis of WWW structure and dynamics. https://doi.org/10.23919/TMA.2019.8784665 (2019).
32. A. Krishna Murthy and Suresha. XML URL classification based on their semantic structure orientation for web mining applications. *Proc. Comput. Sci.* https://doi.org/10.1016/j.procs.2015.02.005 (2015).

33. Ubing, A. A., Jasmi, S. K. B., Abdullah, A., Jhanjhi, N. Z. & Supramaniam, M. Phishing website detection: an improved accuracy through feature selection and ensemble learning. *Int. J. Adv. Comput. Sci. Appl.* https://doi.org/10.14569/IJACSA.2019.0100133 (2019).
34. Aggarwal, A., Rajadesingan, A. & Kumaraguru, P. PhishAri: Automatic realtime phishing detection on twitter. https://doi.org/10.1109/eCrime.2012.6489521 (2012).
35. Prasad, A. & Chandra, S. PhiUSIIL: A diverse security profile empowered phishing URL detection framework based on similarity index and incremental learning. *Computers Secur.* **136**, 103545. https://doi.org/10.1016/j.cose.2023.103545 (2023).
36. Barik, K., Misra, S. & Mohan, R. Web-based phishing URL detection model using deep learning optimization techniques. *J. Intell. Fuzzy Syst.* https://doi.org/10.1007/s41060-025-00728-9 (2025).
37. Uddin, M. A. & Sarker, I. H. An explainable transformer-based model for phishing email detection: A large language model approach. *arXiv* https://arxiv.org/abs/2402.13871 (2024).
38. Li, G., Cui, Y. & Su, J. Adaptive mechanism-based grey Wolf optimizer for feature selection in high-dimensional classification. *PLoS One*. **20** (5), e0318903. https://doi.org/10.1371/journal.pone.0318903 (2025).
39. Thakur, S. Feature selection using adaptive weight based grey Wolf optimization for malware detection in android. *Int. J. Intell. Eng. Syst.* **17** (3), 1–10. https://doi.org/10.11591/ijeecs.v17.i3.pp1-10 (2024).
40. Barik, K., Misra, S. & Mohan, R. Web-based phishing URL detection model using deep learning optimization techniques. *Int. J. Data Sci. Analytics*. https://doi.org/10.1007/s41060-025-00728-9 (2025).
41. Pentapalli, L. S., Salisbury, J. & Riep, J. A gradient-optimized TSK fuzzy framework for explainable phishing detection. *ArXiv* https://doi.org/10.2139/ssrn.4501234 (2025).
42. Kelvin, O. Cloud-secure: An investigation into firefly and grey wolf optimization algorithms for phishing detection with machine learning classifiers. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 2 (SIGCSE 2024). Association for Computing Machinery, New York, NY, USA*. 1887. https://doi.org/10.1145/3626253.3635401 (2024).
43. George, P. & Vinod, P. *Composite Email Features for Spam Identification*. https://doi.org/10.1007/978-981-10-8536-9_28 (Springer, 2018).
44. Hota, H. S., Shrivas, A. K. & Hota, R. An ensemble model for detecting phishing attack with proposed remove-replace feature selection technique. *Proc. Comput. Sci.* https://doi.org/10.1016/j.procs.2018.05.103 (2018).
45. Singh, J. et al. Predicting blood glucose levels in type 1 diabetes using deep learning and regression techniques. In *2025 International Conference on Intelligent Control, Computing and Communications (IC3), Mathura, India*. pp. 86–91, https://doi.org/10.1109/IC363308.2025.10956664 (2025).
46. Mirjalili, S., Mirjalili, S. M. & Lewis, A. Grey Wolf optimizer. *Adv. Eng. Softw.* **69**, 46–61. https://doi.org/10.1016/j.advengsoft.2013.12.007 (2014).
47. https://phishtank.org/.
48. https://archive.ics.uci.edu/ml/datasets/Phishing+Websites
49. Ahsan Wajahat, J. et al. An adaptive semi-supervised deep learning-based framework for the detection of Android malware. *J. Intell. Fuzzy Syst.* **45** (3), 5141–5157 https://doi.org/10.3233/JIFS-231969 (2023).
50. Qureshi, S., Li, J., Akhtar Rajputt, F. & Wajahat, A. Analysis of challenges in modern network forensic framework. *Secur. Communication Networks*. https://doi.org/10.1155/2021/8871230 (2021).
51. Wajahat, A. et al. An effective deep learning scheme for android malware detection leveraging performance metrics and computational resources. *Intell. Decis. Technol.* **18** (1), 33–55. https://doi.org/10.3233/IDT-230284 (2024).
52. Wajahat, A. et al. Outsmarting android malware with Cutting-Edge feature engineering and machine learning techniques. *Computers Mater. Continua*. **79** (1), 651–673. https://doi.org/10.32604/cmc.2024.047530 (2024).
53. Azeem, M. Z. et al. &. Effects of code cloning in mobile applications. In *Proceedings of the 2020 3rd International Conference on Computing, Mathematics and Engineering Technologies (iCoMET)*. https://doi.org/10.1109/iCoMET48670.2020.9073872 (IEEE, 2020).
54. Nazir, A. et al. &. Evaluating Energy Efficiency of Buildings using Artificial Neural Networks and K-means Clustering Techniques. In *Proceedings of the 2020 3rd International Conference on Computing, Mathematics and Engineering Technologies (iCoMET)*. https://doi.org/10.1109/iCoMET48670.2020.9073816 (IEEE, 2020).
55. Safi, A. & Singh, S. A systematic literature review on phishing website detection techniques. *J. King Saud Univ. Comput. Informat. Sci.* **35** (2), 590–611 https://doi.org/10.1016/j.jksuci.2023.01.004 (2023).
56. Haq QEu, Faheem, M. H. & Ahmad, I. Detecting phishing urls based on a deep learning approach to prevent Cyber-Attacks. *Appl. Sci.* **14** (22), 10086. https://doi.org/10.3390/app142210086 (2024).
57. Arathi Krishna, V., Anusree, A., Blessy Jose, K., Anilkumar & Ojus Thomas Lee. Phishing detection using machine learning based URL analysis: A survey. *Int. J. Eng. Res. Technol. (IJERT) NCREIS*. **09** (13), 156–161 (2021).
58. Wang, J., Lin, D., Zhang, Y. & Huang, S. An adaptively balanced grey Wolf optimization algorithm for feature selection on high-dimensional classification. *Eng. Appl. Artif. Intell.* **114**, 105088 (2022).
59. Zhang, Y. & Cai, Y. Adaptive dynamic self-learning grey Wolf optimization algorithm for solving global optimization problems and engineering problems. *Math. Biosci. Eng.* **21** (3), 3910–3943. https://doi.org/10.3934/mbe.2024174 (2024).
60. Ragab, M. et al. Enhanced gravitational search optimization with hybrid deep learning model for COVID-19 diagnosis on epidemiology data. (PMC article) This uses an optimization algorithm (EGSO → Enhanced Gravitational Search Optimization) combined with a deep learning model for diagnosis, showing how meta-heuristic optimization can improve deep network performance. (2022).
61. Qasim, O. S. et al. A new hybrid algorithm based on binary grey Wolf optimization and firefly algorithm for feature selection. *J. Forestry Res.* **14**, 2510172 (2024).
62. Mbura, R. K. et al. A Novel Hybrid Approach for Identification of Discriminative Features in Phishing Emails. *IEEE Access.* **14**, 995–1013 (2026). https://doi.org/10.1109/ACCESS.2025.3649636

Acknowledgements

Author thanks. In most cases, sponsor and financial support acknowledgments.

Author contributions

Monika Dandotiya (Author 1): Conceptualization, Methodology Design, Data Curation, Writing – Original Draft Preparation.Nikhil Goyal (Author 2): Software Implementation, Data Analysis, Visualization, Validation. Ajay Khunteta (Author 3): Supervision, Guidance on Research Framework, Critical Review, and Editing.Babita Tiwari (Author 4):Supervision, Proposed Framework and discussion.

Funding

Open access funding provided by Manipal University Jaipur.

Declarations

Competing interests
The authors declare no competing interests.

Additional information
Correspondence and requests for materials should be addressed to B.T.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

© The Author(s) 2026